Cacophony

The Manual

A synthetic reality compiler. Schemas in, worlds out — and the same world every time you ask for it.

VERSION 0.1.0 · COMPLETE EDITION · FOURTEEN PHASES, DELIVERED

About this manual

This is the whole of Cacophony, written down. Every command, every option, every generator, every key the schema language accepts, and the reasoning behind the parts where the reasoning is the interesting bit.

It is long on purpose. Cacophony is a compiler for realities, and a compiler that will happily produce four hundred million rows deserves a reference you can look something up in at two in the morning without guessing. Where a chapter is a tutorial it says so; everywhere else, assume you are reading reference material and skip freely.

How it is arranged

Twelve parts and a stack of appendices. Part I gets a dataset out of the machine in ten minutes. Part II explains the two or three ideas the whole system rests on — principally that a record's seed is a hash of its *position*, which is why nearly everything else is cheap. Parts III to VI are the schema language. Parts VII to IX are running, scaling and checking. Parts X to XII are the interfaces, the extension points and the operational detail. The appendices are the tables you will actually use.

Conventions

Command lines and file contents are set in monospace and labelled with their language. A label of *terminal* means you type it; the `$` is the prompt, not part of the command.

References like section 75 point at the numbered sections of `CACOPHONY.md`, the design document this program was built from. They appear throughout because a good deal of Cacophony's behaviour is a deliberate reading of a specific requirement, and the requirement is often the shortest explanation of the behaviour.

British spelling throughout, matching the source. Where a schema key or a command differs — `normalize`, `--no-validate` — the code spelling wins, because that is what you have to type.

> **TWO WORDS THAT MEAN DIFFERENT THINGS**
>
> **Chaos** is damage: data the schema forbids, injected on purpose — a null in a required column, a key pointing at nothing. **Edge cases** are legal values that naive code mishandles anyway — `O'Brien-Smith`, an emoji in a display name, the year's last second. The first tests your error handling; the second tests your correctness. They are separate features with separate flags, and conflating them is the most common way to misread this manual.

Versions

Written against Cacophony **0.1.0**, the state of the repository after the fourteenth and final delivery phase. Of the design document's 112 sections, 86 are built, with 25 partial and 1 deferred; 6 things it asks for were considered and refused. `docs/CONFORMANCE.md` is that list, generated rather than asserted. Every command in this manual was run against that build while the manual was being written; the outputs shown are real, including the ones that report a problem.

Licence

Cacophony is free software under the GNU Affero General Public License, version 3 or later. The Affero clause is the one that matters for a program like this — it has a web interface and a distributed mode, so it is the kind of thing people run on a server for other people to use, which is precisely the case ordinary GPL leaves open.

The data you generate is yours. The licence says nothing about it, and a schema is your document rather than a derived work of the program.

Set in IBM Plex — Serif for reading, Sans for structure, Mono for anything the computer said. Composed from HTML and CSS with WeasyPrint; the sources are in `docs/manual/` and the build is one command.

Contents

PART I

Beginning

What this program is, how to install it, and a dataset on disk before the kettle boils.

CHAPTER

1

What Cacophony is

Cacophony is a compiler. You write a schema describing a world — people, machines, tickets, transactions, the shape of a working week — and it produces the records that world would have generated, in the formats your systems already read, reproducibly, at whatever scale you asked for.

That framing matters more than it sounds. A fake-data library gives you a function that returns a plausible name. Cacophony gives you a compilation unit: a project file that is the single source of truth for an entire dataset, from which the dataset can be re-derived exactly, in whole or in part, on any machine, at any time, by anyone holding the file.

1.1 The problem it exists for

Everybody who builds software eventually needs data that looks real and is not. Demonstrations need a populated database. Test suites need fixtures with awkward corners in them. Machine-learning work needs volume. Security tooling needs a year of authentication logs with something interesting buried in the middle. Load tests need a workload, not a file.

The usual answers are all unsatisfying in the same way. Copying production data is a compliance incident waiting for a date. Hand-written fixtures are tiny, stale and unrepresentative. A loop around a fake-data library gives you a million rows of noise: names that belong to no one, orders that reference no customer, timestamps distributed uniformly across a year in which nobody sleeps. Each row is plausible; the dataset is not.

Cacophony's answer is to treat the dataset as the output of a program you write once and can run forever:

- **Structure comes first.** Entities have fields, fields have types and constraints, and references between entities are real — the foreign keys resolve, and a database output enforces them.

- **Meaning drives generation.** A field says what it *means* in plain language, and the right strategy is chosen from that: a pattern, a distribution, a lookup, or a language model when the field is genuinely prose.
- **Time is a first-class thing.** Events fall on a timeline with a shape — quiet at night, busy on Monday, dead on the holiday you listed — and a subject's history accumulates state.
- **Everything is reproducible.** The same schema and the same seed produce the same bytes on any machine, whether generated by one process or forty across eight hosts.

1.2 What comes out

CSV, JSON, JSON Lines, Parquet, a SQLite database with enforced foreign keys, or a portable SQL script. Images, speech and PDF documents when the schema asks for them, written beside the records with a manifest that ties each file to the row it belongs to. A live stream to syslog, HTTP, Kafka, a rotating file or standard output when what you need is a workload rather than a dataset.

1.3 What it will not do

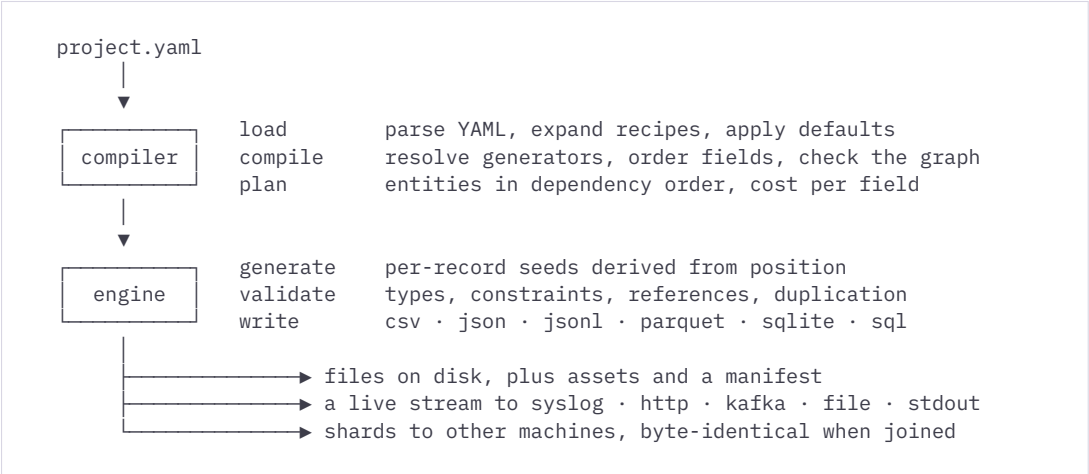
A manual that only lists capabilities is a brochure. Four things Cacophony deliberately does not do, each for a reason set out later in these pages:

- **It will not run code from a project file.** The `script` generator is declared in the design and deliberately not implemented, because a schema is something people send each other and opening one must never be equivalent to running its author's code. Chapter 41 gives the full reasoning, including what isolation was actually measured before the decision was taken.
- **It will not produce identifiers that could belong to somebody.** Email domains, IP addresses, MAC prefixes, telephone numbers and government identifiers are drawn from the ranges the standards bodies reserved for documentation and fiction. This is on by default and switchable per field.
- **It will not hold a credential.** A project names a *logical* secret; the value is resolved at run time from the environment or the OS keychain, and the loader rejects anything that looks like a literal key.
- **It will not pretend a language model is trustworthy.** Model output is extracted, parsed, type-checked, constraint-checked, repaired where repair

needs no judgement, and retried up a three-step ladder. Then it stops and tells you.

1.4 The shape of the thing

One Python package with a command-line interface, an HTTP API, a React front-end called the Studio, and a Tauri desktop shell that hosts the Studio with the backend as a child process. The same application serves all of them: there is no desktop mode and no separate server build, which is the only reliable way to keep three front doors from drifting apart.



The path from a file to a dataset. Every stage is inspectable from the command line: `validate`, `lint`, `plan`, `prompt`, `preview`.

1.5 Who it is for

Engineers building the systems that will eat the data, and the people who have to trust what comes out: test engineers who need fixtures that break things honestly, data engineers who need volume with correct relationships, security teams who need a year of logs with an incident in them, and anyone who has been told they cannot have a copy of production and were right to be told so.

CHAPTER

2

Installing

Cacophony needs Python 3.12 or newer and nothing else to generate data. Everything beyond that — the API, Parquet, Kafka, a language model — is an optional extra you install when you want it.

2.1 From the repository

TERMINAL

```
$ git clone git@github.com:jeddhor/Cacophony.git
$ cd Cacophony
$ python -m venv .venv
$ .venv/bin/pip install -e '.[api]'
$ .venv/bin/cacophony version
```

The editable install puts a `cacophony` executable in the virtualenv's `bin/`. Everything in this manual assumes it is on your path; if it is not, spell it out as `.venv/bin/cacophony` or run the package directly with `python -m cacophony`, which is the same program by another door.

2.2 Extras

OPTIONAL DEPENDENCY GROUPS

EXTRA	BRINGS	NEEDED FOR
`api`	FastAPI, uvicorn	`cacophony serve` and `cacophony desktop` — the server that *hosts* the Studio; the Studio itself is built with Node, below
`parquet`	pyarrow	the `parquet` output format
`kafka`	a Kafka client	`--to kafka://…` when streaming
`keyring`	keyring	resolving a provider secret from the OS keychain section 63
`dev`	pytest, ruff, mypy, httpx2	working on Cacophony itself

TERMINAL

```
$ .venv/bin/pip install -e '.[api,parquet]'
```

2.3 The Studio

The web front-end is built with Node and served by the same process that serves the API. It is built once and lives inside the package:

TERMINAL

```
$ npm --prefix frontend install
$ npm --prefix frontend run build     # writes into backend/cacophony/api/
static/
$ cacophony serve                     # http://127.0.0.1:8765
```

A FRESH CLONE HAS NO STUDIO

The built front-end is generated output and is not committed, so `cacophony serve` on a fresh clone offers the API and no interface until `npm run build` has run. This is deliberate: a build artefact in version control is a build artefact that goes stale.

2.4 A language model, if you want prose

Nothing in Cacophony requires a model. Names, addresses, timestamps, identifiers, distributions and references are all deterministic and local. A model is needed only for fields that are genuinely prose — a support ticket's resolution notes, a product description, a biography.

Any OpenAI-compatible server works, as does Ollama or llama.cpp directly. The provider is named in the project file; see chapter 26. To try the machinery without a server at all, use the `mock` adapter, which synthesises answers from the schema and can be told to fail on purpose.

2.5 Checking the installation

TERMINAL

```
$ cacophony version
$ cacophony generators              # 30 strategies, with aliases and cost
class
$ cacophony recipes                 # 31 reusable schema fragments
$ cacophony validate templates/corporate-directory.yaml
```

The repository ships eight complete example projects in `templates/`. They are not toys — each one compiles, generates and validates in the test suite — and reading them is the fastest way to learn the schema language. Chapter 24 describes each.

CHAPTER

3

Ten minutes to a dataset

A TUTORIAL. BY THE END OF IT YOU WILL HAVE A SCHEMA WITH TWO RELATED ENTITIES, A preview on screen, a hundred thousand records on disk, a SQLite database whose foreign keys resolve, and a clear idea of which command answers which question.

3.1 A first schema

Put this in `company.yaml`. It is complete; nothing is elided.

YAML

```yaml
project:
  name: company
  description: A small software company, invented.
  seed: 20260816

entities:
  employee:
    count: 250
    primary_key: employee_id
    fields:
      employee_id:
        type: string
        generator: sequence
        format: "EMP-{000000}"
        primary_key: true
        unique: true
      first_name:
        type: string
        generator: faker
        provider: first_name
      last_name:
        type: string
        generator: faker
        provider: last_name
      email:
        type: email
        generator: template
        template: "{first_name|lower}.{last_name|lower}@example.com"
        unique: true
      department:
        type: enum
        generator: weighted
        choices:
          Engineering: 45
          Sales: 20
          Support: 18
          Finance: 9
          People: 8
      salary:
        type: integer
        generator: distribution
        distribution: lognormal
        mean: 11.2
        stddev: 0.35
        min: 42000
        max: 310000
      started_on:
        type: date
        generator: datetime
        start: "2019-01-01"
        end: "2026-06-30"
```

3.2 Does it compile?

```
TERMINAL

$ cacophony validate company.yaml

CACOPHONY validate
company
──────────────────────────────────────────────────────────────────────────────
schema is valid  company.yaml
  entities        1
  records         250
  field values    1,750
  entity order    employee
```

`validate` answers a structural question: does this file parse, do the generators exist, do the options they were given make sense, can the fields be ordered so that everything a field depends on is produced before it. It does not generate anything.

3.3 Is it a good idea?

```
TERMINAL

$ cacophony lint company.yaml

CACOPHONY lint
company
──────────────────────────────────────────────────────────────────────────────
no issues found
```

`lint` is the second opinion: a set of rules about schemas that compile but will disappoint you. Nineteen rules at three severities, listed in appendix I. `--strict` makes warnings fatal, which is what you want in continuous integration.

3.4 What will it do?

```
TERMINAL

$ cacophony plan company.yaml

CACOPHONY plan
company
──────────────────────────────────────────────────────────────────────

seed 20260816

Generate 250 employee
     employee_id       sequence(EMP-{000000})
     first_name        faker(first_name)
     last_name         faker(last_name)
     email             template({first_name|lower}.{last_name|lower}@example.com)
     department        weighted(Engineering, Sales, Support, +2 more)
     salary            distribution(lognormal)
     started_on        datetime(2019-01-01..2026-06-30)

──────────────────────────────────────────────────────────────────────

  records            250
  field values       1,750
  language-model     0 calls
  storage (approx)   37.6 KB
```

The plan is the compiled truth: which generator each field ended up with, what it costs, and what it waits for. `email` is ordered after the two names because the template mentions them — you did not declare that and you cannot get it wrong.

3.5 Look at some records

```
TERMINAL

$ cacophony preview company.yaml -e employee -n 3 \
      -c employee_id,first_name,last_name,email,department

                              employee  (3 records)

┌─────────────┬────────────┬───────────┬──────────────────────────┬─────────────┐
│ employee_id │ first_name │ last_name │ email                    │ department  │
├─────────────┼────────────┼───────────┼──────────────────────────┼─────────────┤
│ SEQ         │ FAKER      │ FAKER     │ TMPL                     │ WEIGHT      │
│ EMP-000001  │ Cynthia    │ Decker    │ cynthia.decker@example.com │ Sales     │
│ EMP-000002  │ Michelle   │ Hall      │ michelle.hall@example.com  │ Engineering │
│ EMP-000003  │ Douglas    │ Braun     │ douglas.braun@example.com  │ People    │
└─────────────┴────────────┴───────────┴──────────────────────────┴─────────────┘

all sampled records passed validation
```

The row of abbreviations under the header names the generator behind each column, which is the quickest way to spot a field that quietly got a generator you did not intend.

> **PREVIEW SHOWS THE REAL RECORDS**
>
> Not a sample drawn from somewhere nearby — the actual records that a real run would write at those indices. Because a record's seed is derived from its position rather than from a shared random stream, previewing costs nothing and disturbs nothing. Run it a hundred times; the production dataset is unaffected, because there is no shared state to disturb. Use `--offset 4823913` to look at a record in the middle of a run you have not done yet.

3.6 Add a second entity, with a real reference

YAML

```yaml
ticket:
  count: 5000
  primary_key: ticket_id
  fields:
    ticket_id:
      type: string
      generator: sequence
      format: "TIC-{000000}"
      primary_key: true
      unique: true
    raised_by:
      generator: reference
      entity: employee
      distribution: skewed
      skew: 1.9
    opened_at:
      type: datetime
      generator: datetime
      start: "2026-01-01"
      end: "2026-06-30"
    status:
      type: enum
      generator: weighted
      choices: {resolved: 70, in_progress: 18, waiting: 8, escalated: 4}
    reporter_email:
      type: email
      generator: template
      template: "{employee.email}"
```

Three things happened there, and all three are worth noticing.

The `raised_by` field is a foreign key that resolves: it holds a real `employee_id`, and it is *skewed*, so a tenth of the staff raise about a quarter of the tickets, which is what support queues actually look like. Uniform references are the single most common way synthetic data stops resembling anything.

The `reporter_email` template reads a field of the referenced employee. Cacophony reconstructs that employee from its index at the moment the tem-

plate needs it — no pool of parents is held in memory, which is why five million tickets pointing at two hundred and fifty employees costs the same as five.

And `ticket` is now ordered after `employee` automatically. You did not write a `relationships:` block; the reference is the relationship.

3.7 Generate

TERMINAL

```
$ cacophony generate company.yaml -o jsonl -d out/

  employee ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 250/250   3,507/s  0:00:00
  ticket   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5000/5000 14,353/s 0:00:00
──────────────────────────────────────────────────────────────────────────
complete  5,250 records in 0.58s
  throughput      9,128 records/sec
  files           2
  run             64c7c9fa-4092-4eb8-91dc-5aa3e4e5b429
  referential     100.00%  (5,000 references checked)
  distributions   97.68% match
  references      10,000 resolved, 98% from cache
```

-N OVERRIDES EVERY ENTITY

`--records 100000` would give you a hundred thousand employees *and* a hundred thousand tickets, not a hundred thousand tickets raised by two hundred and fifty people. It is a blunt instrument for smoke tests. To change one entity, change its `count:`.

It would also stop: a hundred thousand employees cannot have distinct addresses drawn from a list of first and last names, and `email` is declared `unique`. That is the run doing its job — chapter 32 explains what to do about it.

TERMINAL

```
$ head -1 out/ticket.jsonl
{"ticket_id": "TIC-000001", "raised_by": "EMP-000079",
 "opened_at": "2026-03-03T06:35:46.305373", "status": "resolved",
 "reporter_email": "steven.schmidt@example.com"}
```

3.8 Prove the relationships hold

TERMINAL

```
$ cacophony generate company.yaml -o sqlite -d out/
$ sqlite3 out/company.db "PRAGMA foreign_key_check"      # silent: nothing
broken
$ sqlite3 out/company.db \
      "SELECT department, COUNT(*) FROM employee GROUP BY 1 ORDER BY 2 DESC"
Engineering|103
Support|51
Sales|51
People|24
Finance|21
```

The SQLite output writes one database for the whole project, every entity a table, every reference an enforced `FOREIGN KEY`. Silence from `foreign_key_check` is the point: the data is relationally sound, not merely shaped like it.

3.9 Stop it and start it again

Press `Ctrl-C` halfway through a large run and then:

TERMINAL

```
$ cacophony runs
$ cacophony resume 4f2a91c3
```

The run is checkpointed after every batch. On resume the file on disk is counted and the checkpoint corrected to match it, so an unclean stop cannot leave you with duplicated or missing records. The resumed run reuses the original seed, format and provenance mode, because a dataset generated two different ways is not one dataset.

3.10 What to read next

WHERE TO GO FROM THE TUTORIAL

IF YOU WANT TO…	READ
Understand why previewing and resuming are free	Chapter 4, determinism
Know every field key and generator option	Parts III and IV
Make events happen on a realistic timeline	Chapter 18
Bury an incident in a year of logs	Chapter 19
Break the data on purpose	Chapters 20 and 34

IF YOU WANT TO…	READ
Stop writing the same twelve fields	Chapter 21, recipes
Generate faster than one machine can	Chapter 30
Send somebody the whole project	Chapter 23, bundles
Describe a world in a sentence and build it	Chapter 28

PART II

How it thinks

Three ideas carry most of the system: a seed derived from position, a compile that happens before anything is generated, and a refusal to invent identifiers that could belong to somebody real.

CHAPTER

4

Determinism and the seed tree

If you understand one chapter of this manual, make it this one. Nearly every convenience elsewhere — free previews, cheap resume, memoryless references, parallel generation with identical bytes, regenerating a single row a year later — is a consequence of one decision about where randomness comes from.

4.1 The usual way, and why it fails

The obvious design is a seeded random number generator threaded through generation. Seed it once, and the sequence of values is reproducible from the start.

It is reproducible only *from the start*, and only *in the same order*. That single property costs you everything interesting. Record 4,823,913 cannot be produced without producing the 4,823,912 before it. Previewing a sample advances the stream, so the preview changes what the run would have written. Two threads generating two entities interleave their draws, so the output depends on scheduling. Resuming needs the RNG's internal state serialised to disk and restored exactly. Generating on eight machines is out of the question.

4.2 What Cacophony does instead

A record's seed is a hash of its *position*. Nothing is carried forward; every seed is computed from the coordinates of the value being produced.

```
project seed          20260816
      │
      ├── entity "employee"          mix(project, "entity", hash("employee"))
      │         │
      │         ├── record 0          mix(entity, "record", 0)
      │         ├── record 1          mix(entity, "record", 1)
      │         └── record 4823913    mix(entity, "record", 4823913)
      │                   │
      │                   ├── field "first_name"   mix(record, "field", hash(name))
      │                   ├── field "salary"
      │                   └── field "portrait"
      │                             │
      │                             └── draw 0, 1, 2 …   mix(field, "draw", k)
      │
      │
      └── entity "ticket"           …and so on, independently
```

The seed hierarchy of section 75. Each level is derived from its parent and its own coordinate, so any node can be computed directly without computing its siblings.

The derivation uses BLAKE2b at the root and a SplitMix64 mixing step for each level below it. The details are not the point; the shape is. Because `mix(parent, salt, component)` is a pure function, the seed for *employee, record 4,823,913, field salary* can be computed on its own, on any machine, in any order, at any time, with nothing else in memory.

4.3 What that buys

CONSEQUENCES OF POSITIONAL SEEDING

PROPERTY	WHY IT FOLLOWS
Previews cost nothing and disturb nothing	There is no shared stream to advance. `preview --offset N` shows exactly what a run would write at N.
Resume needs one integer	A checkpoint is a count of finished records. There is no RNG state to serialise, so no way for it to be stale or corrupt.
References cost no memory	A parent is reconstructed from its index rather than held in a pool. A hundred million events pointing at five thousand employees is the same state as five.
Parallelism is free and exact	A shard is an index range, and its output is a pure function of that range. Forty workers produce what one would have, byte for byte.
A failed shard is redone, not repaired	The second attempt produces exactly the bytes the first would have, so a dead worker's partial file is simply overwritten.
Regeneration is nearly free	`cacophony regenerate -r 4823913` answers "is this row wrong" in milliseconds, with no dataset and no run.
Edge cases and chaos are reproducible	Which record gets damaged, and how, is derived from its index. A bug found once is found again.
Streams continue rather than repeat	`--from N` resumes an endless stream at the next index instead of replaying the first one.

4.4 Where the world intrudes

Determinism is a property of the deterministic generators, which is twenty-six of the thirty. Four cannot promise it, and say so:

- `llm` — a language model on a server you do not control. The prompt, the seed and the request are identical every time; the answer may not be. The generation cache exists partly for this reason: with `--cache read_write`, an answer once obtained is reused.
- `image` and `tts` — the same, with a GPU attached. The *procedural* adapters for both are fully deterministic and need no server, which is what makes a multimodal schema designable on a laptop.
- `document` — renders in-process from the record, but a PDF embeds a creation timestamp.

`cacophony plan` marks every field with its determinism and cost class, and the distributed controller reads shard requirements off exactly the same information. Nothing here is declared twice.

4.5 Seeds you can set

WHERE	EFFECT
`project.seed`	The root. Everything descends from it.
`entities.<name>.seed`	Detaches one entity, so it can be regenerated independently of changes elsewhere.
`--seed N`	Overrides the project seed for one command. The same schema with two seeds is two populations.
`worlds`	A name for a seed plus the schema it goes with, so today's logins and next week's tickets describe the same company. Chapter 22.

> **PIN YOUR SEED**
>
> Leaving `seed` unset still produces deterministic output — the default is zero, not a clock reading — but a project that states its seed says so to the next reader, and to the reviewer wondering whether the dataset in the ticket is the one they are looking at.

CHAPTER

5

The pipeline, end to end

Between a project file and a row on disk there are seven stages. Each is separately inspectable from the command line, which is the difference between a system you can debug and one you can only rerun.

5.1 Load

The YAML or JSON is parsed into a specification model — typed, with defaults applied and unknown *structural* keys refused. Unknown keys on a *field* are not refused: they become generator options, which is what makes `generator: sequence` plus `format: "EMP-{000000}"` work without ceremony.

Relative paths in a project resolve against the **schema file**, not the working directory. Without that rule a project only works from the directory it lives in, and a portable bundle is impossible.

5.2 Expand

Recipes expand here, before anything else looks at the schema. A recipe's fields become ordinary fields; nothing downstream knows a recipe existed except the `recipe:` attribution each expanded field carries for the benefit of `plan` and the Studio. That ordering is deliberate: validation, linting, planning and generation all see one kind of field.

5.3 Compile

Generators are resolved by name or alias, instantiated with their options, and asked to validate those options immediately — so a misspelled option is an error from `cacophony validate` rather than a surprise three million records into a run. Field dependencies are collected from three places: what the generator declares, what a template or expression mentions, and whatever `depends_on` adds by hand. The result is a graph.

5.4 Plan

The graph is topologically sorted — fields within an entity, entities within the project — and a cycle is an error naming the cycle. The plan records, per field, its generator, cost class (`cpu`, `llm`, `gpu`), determinism, and what it waits for. `cacophony plan --json` is that structure verbatim.

5.5 Generate

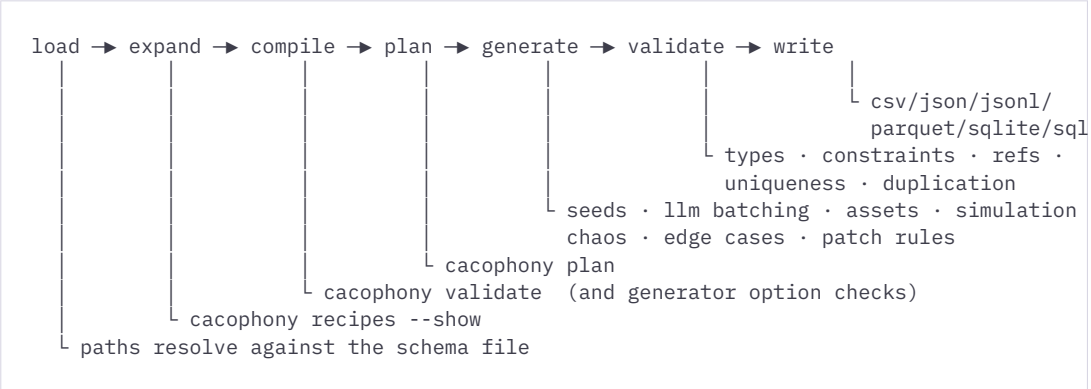
Records are produced in batches. Within a record, fields are produced in plan order, each with its own derived seed. Language-model fields may be batched across records to cut call counts; media fields are content-addressed so identical bytes are written once. Simulation folds state over a subject's contiguous block of events. Chaos damages a fraction of values. Edge cases substitute awkward-but-legal ones. Patch rules apply last, before validation, so what the validator checks is what reaches the file.

5.6 Validate

Types, constraints, uniqueness, referential integrity and — when the schema asks — duplication. Deliberately damaged fields are skipped, since a chaotic run would otherwise be a wall of failures reporting the feature working. Policy is `--on-failure`: `abort` (default), `retry`, `skip`, `placeholder` or `incomplete`; `--drop-invalid` discards failures silently and `--no-validate` switches the stage off.

5.7 Write

Writers consume bounded batches and flush them. Nothing accumulates: a dataset far larger than memory costs the same as a small one. A checkpoint is recorded after each batch, and the run store gets an event.



Seven stages, five of them with a command that prints what they decided.

5.8 Inspecting each stage

QUESTION	COMMAND
Does this file make sense at all?	`cacophony validate`
Is it a sensible design?	`cacophony lint`
What did the recipe add?	`cacophony recipes --show NAME`
Which generator did each field get, and in what order?	`cacophony plan`
What exactly is being sent to the model?	`cacophony prompt`
What will the records look like?	`cacophony preview`
What did that run actually do?	`cacophony run ID --events 50`
Why is this one row wrong?	`cacophony regenerate -r N`

CHAPTER

6

Safety, and the fiction

Synthetic data has two ways of hurting somebody. It can contain a real person's details, or it can be mistaken for a real observation. Cacophony takes both seriously by default, and both are switchable per field, because a default that cannot be turned off gets worked around.

6.1 Identifiers that cannot belong to anyone

Every generator that produces a real-world identifier draws from the range the relevant standards body reserved for documentation and fiction section 62. This is on by default and controlled by `safe:` on the field.

RESERVED RANGES USED BY DEFAULT

KIND	DRAWN FROM	AUTHORITY
Email domains	`example.com`, `example.org`, `example.net`, `*.example`	RFC 2606
IPv4	`192.0.2.0/24`, `198.51.100.0/24`, `203.0.113.0/24`	RFC 5737
IPv6	`2001:db8::/32`	RFC 3849
MAC addresses	the documentation OUI	RFC 7042
Telephone numbers	`555-0100` to `555-0199`	the North American fiction block
Government identifiers	the never-issued 900 area	US SSA policy
Hostnames	documentation domains	RFC 2606
CVE identifiers	a year with none published	—

The Faker generator is included in this: values that carry a domain are rewritten onto reserved domains unless the field says `safe: false`. Turning it off is a deliberate act on one field, which is the right size for that decision — a schema needing genuinely-valid-looking addresses for a geocoder test can have them without the whole project losing its guarantees.

> **THE CVE CASE**
>
> The `cve_identifier` recipe produces identifiers shaped exactly like real ones, in a year for which none have been published. This is deliberate. Synthetic security data that used real CVE numbers would, the moment it leaked into a ticket or a report, be indistinguishable from threat intelligence about a real vulnerability.

6.2 Detectors for what got through

Reserved ranges stop Cacophony's own generators producing anything real. They do not stop a language model reproducing something it saw, which is what section 61 is about, so the detectors are the second line:

```yaml
privacy:
  policy: warn      # report, warn, or block
```

Card numbers that pass a Luhn check, government identifiers in issuable ranges, domains outside the reserved names, routable addresses, dialable telephone numbers. Absent means not asked for; `block` makes a finding a validation failure, so `--on-failure` decides what happens next.

Where each one looks is deliberate, and worth knowing before you rely on it. Card numbers and government identifiers are found anywhere in a value. Domains are found inside an email address or a URL anywhere in a value, and a bare hostname is reported when the field is declared to hold one — `hostname`, `uri`, `email` or `domain`. IPv4 addresses and telephone numbers must be the whole value, unless the field's type says it carries one, in which case they are found anywhere inside it. IPv6 addresses count anywhere, since nothing else is shaped like one. Every candidate is parsed before it is reported.

The reason for those boundaries is the failure mode of the alternative: a detector that scans free prose for dotted quads reports `rv:1.9.6.20` in every user agent it ever sees, and one that reports every dotted word reports `config.yaml`. A detector people switch off finds nothing at all.

> **WHAT THEY FOUND ON THEIR FIRST RUN**
>
> Turned on the shipped templates, they immediately caught `corporate-directory.yaml` emitting real-shaped, dialable telephone numbers: Faker's safe mode rewrote domains and nothing else, so `provider: phone_number` walked straight past it. Faker's phone and identifier output is now replaced from the same reserved ranges the dedicated generators use, and every template is checked by a test.

> **THIS IS LEAK DETECTION, NOT PRIVACY**
>
> Section 61 says not to claim synthetic output is privacy-preserving unless an actual privacy technique is used, and detection is not one. It catches things that look real. It says nothing about what could be inferred from a dataset as a whole, and Cacophony does not claim otherwise.

6.3 Credentials are never in the file

A provider names a *logical* secret. The value is resolved at run time section 63:

```yaml
providers:
  hosted_llm:
    type: language_model
    adapter: openai_compatible
    base_url: https://models.internal.example/v1
    model: qwen3-8b
    secret: models-internal        # a logical id, never a value
```

1. The environment variable `CACOPHONY_SECRET_MODELS_INTERNAL`.
2. An environment variable named after the id itself.
3. The OS keychain, under the service name `cacophony`.

The loader rejects a `secret:` whose value looks like a literal key rather than an identifier. The distributed controller's shared token is read from `CACOPHONY_CONTROLLER_TOKEN` for the same reason, rather than taken as a flag — a flag lands in shell history and in every process listing on the box.

6.4 Untrusted input is treated as such

Three places take input from outside and all three assume the worst.

- **Bundles.** Import refuses path traversal, absolute paths, Windows drive letters and symlink entries, applies size and entry-count ceilings, and checks the whole archive before writing a single byte — so one bad entry leaves nothing behind rather than half a project.
- **Expressions.** Both the `expression` generator and the `--where` filters are evaluated from a parsed syntax tree against an allow-list of node types and functions. No `eval`, no imports, no attribute access beyond dotted record lookups.

- **Plugins.** Discovered through installed entry points and never from a project directory. The trust decision belongs to the person running `pip install`, not to a program opening a file somebody emailed them.

6.5 What a served Cacophony can reach

The API registers projects by path, rewrites their schemas, and writes runs where the caller says. On loopback that is exactly as powerful as the shell that started it, which is the honest description of a local tool. On any other interface it is somebody else's shell, so a token and a set of permitted directories are required before it will start — and a path outside them is refused with a 403 rather than quietly resolved.

6.6 Provenance, and what it retains

`--provenance` records how data came to exist. The highest level retains prompts and raw provider responses, which may contain content you would rather not keep, so it is always an explicit opt-in section 87.

PROVENANCE MODES

MODE	RECORDS
`none`	Nothing. The default.
`run`	Run-level facts: schema hash, seed, versions, timings.
`record`	Per record: its index, its seed, the run that made it.
`field`	Per field: the generator, its seed, whether chaos touched it.
`full`	The above plus prompts and raw model responses. Opt in knowingly.

PART III

The schema language

One file, up to eleven top-level keys, and a field model that turns anything it does not recognise into an option for the generator.

CHAPTER

7

The project file

A Cacophony project is a single YAML or JSON document. Two keys are required and the rest are opt-in, so the smallest useful project is about six lines and the largest shipped example is under three hundred.

7.1 The top level

```yaml
project:        # metadata, seed, locale, provenance mode      (required)
entities:       # the record types to generate                 (required)
relationships:  # documentation of connections between entities
providers:      # generation backends: models, image, speech
timeline:       # when this project's events happen
scenarios:      # behavioural overlays applied to a fraction of subjects
chaos:          # deliberate damage
quality:        # duplication thresholds worth measuring
recipes:        # this project's own reusable fragments
requires:       # what the project needs that it does not carry
patches:        # edits recorded as rules
outputs:        # named output profiles
```

7.2 `project`

PROJECT KEYS

KEY	TYPE	DEFAULT	MEANING
`name`	string	*required*	Project name. Used for the SQLite filename and in the run store.
`description`	string	–	Free text.
`version`	integer	`1`	Schema revision, recorded in provenance.
`seed`	integer	`0`	Root of the seed hierarchy.
`locale`	string	`en_US`	Default Faker locale; also stated to a language model.
`profile`	enum	`balanced`	`quick_mock`, `balanced`, `high_realism`, `maximum_chaos`
`provenance`	enum	`none`	`none`, `run`, `record`, `field`, `full`

WHAT PROFILE DOES

PROFILE	EFFECT
balanced	Nothing. The baseline.
quick_mock	Asks a language model for short, plain answers.
high_realism	Asks a language model for specific, plausible detail over generic phrasing.
maximum_chaos	Turns on the hostile_qa chaos preset and 5% edge cases.

> **THREE OF THE FOUR ONLY TALK TO THE MODEL**
>
> Section 77 describes profiles as selecting how much realism you want, and what is built is narrower than that: the first three change only the prompts a language model is given, not which generators are chosen. Only `maximum_chaos` changes the data, and anything you state explicitly — a `chaos:` block, `--edge-cases` — still wins over it. If you want messier data, write `chaos:` ; if you want less of it, write `count:` .

7.3 entities

YAML

```yaml
entities:
  employee:
    count: 10000
    description: A person employed by the company.
    primary_key: employee_id
    seed: 991                 # optional: detach this entity's seed
    tags: [core]
    recipes: [employee]       # expand a catalogue fragment
    simulation: { ... }       # turn these records into a history
    fields:
      ...
```

ENTITY KEYS

KEY	MEANING
count	How many records. `generate --records N` overrides every entity.
primary_key	Which field identifies a record. May also be set with `primary_key: true` on the field.
seed	Detaches this entity from the project seed, so editing another entity cannot change these records.
description	Free text; offered to a language model as context about the record type.
tags	Free labels, shown by `cacophony plan` and carried in its JSON.
recipes	Catalogue fragments to expand into this entity's fields. Chapter 21.
simulation	Declares these records to be events belonging to subjects. Chapter 18.
fields	The fields, in the order they will appear in the output.

CHAPTER

8

Fields

A field says what it means, what type it is, what values are acceptable, and — optionally — how to produce them. Leave the generator out and one is recommended from the meaning and the type.

8.1 The full form

YAML

```yaml
fields:
  employee_id:
    type: string
    semantic: "The employee's unique staff number"
    description: "Documentation for humans."
    generator: sequence
    format: "EMP-{000000}"       # an unrecognised key: a generator option
    primary_key: true
    unique: true
    nullable: false
    null_probability: 0.0
    depends_on: []
    context: []
    examples: []
    tone: null
    locale: null
    constraints:
      pattern: "^EMP-\\d{6}$"
```

FIELD KEYS

KEY	MEANING
`type`	One of the 28 types below. Default `string`.
`semantic`	What the field **means**, in natural language. Drives generator recommendation and is the backbone of a compiled prompt.
`description`	Documentation. Falls back to `semantic` when that is absent.
`generator`	Which strategy produces the value. Omit it to have one chosen.
`unique`	Enforced by the validator. The linter rejects a domain too small to satisfy it.
`nullable`	`true` alone means 5% null.

KEY	MEANING
`null_probability`	An explicit fraction, 0.0–1.0.
`depends_on`	Extra dependencies beyond those the generator declares. Rarely needed.
`context`	Sibling fields or related entities a generator may consult. For `llm`, what the model is shown.
`primary_key`	Marks the record identifier.
`constraints`	What counts as a valid value. See below.
`examples`	Sample values, shown to a model as exemplars.
`tone`	A style instruction for prose fields.
`locale`	Overrides the project locale for this field.

8.2 Unrecognised keys are generator options

This is the single most important convenience in the schema language and the one most likely to confuse. A key the field model does not recognise is passed to the generator:

```yaml
  hostname:
    type: hostname
    generator: pattern
    pattern: "srv-{a-z:3}-{0000}"     # 'pattern' is an option of 'pattern'
    upper: false                      # so is this
```

The fully explicit form means the same thing and is available when the flat form would be ambiguous:

```yaml
  hostname:
    type: hostname
    generator:
      type: pattern
      pattern: "srv-{a-z:3}-{0000}"
```

OPTIONS ARE NOT CONSTRAINTS

`min:` and `max:` written at the top level of a field are options for the generator; written under `constraints:` they are rules the validator enforces. They usually agree, and when they do not, the validator wins and the generator gets blamed. If you mean "no salary above 310,000 may exist", write it under `constraints`. If you mean "draw from this range", write it as an option. Writing both is the clearest.

8.3 Types

Twenty-eight of them. A type constrains what counts as a valid value and what a writer does with it; it does not decide how the value is produced.

THE TYPE VOCABULARY

GROUP	TYPES
Text	`string` · `text` · `enum`
Numeric	`integer` · `float` · `decimal`
Logical	`boolean`
Identity	`uuid`
Temporal	`date` · `time` · `datetime` · `duration`
Structured	`array` · `object` · `json`
Binary and media	`binary` · `image` · `audio` · `file`
Network	`ip_address` · `cidr` · `mac_address` · `hostname` · `uri`
Contact	`email` · `phone`
Spatial	`geo_point`
Escape hatch	`custom`

Three of these change behaviour elsewhere and are worth calling out. `text` marks a field as long-form prose, which makes it a default candidate for duplication measurement. `decimal` is carried as a decimal all the way to the writer, so money does not acquire binary floating point error on its way to a CSV. `image`, `audio` and `file` put a path in the record and the bytes under `assets/`.

8.4 Constraints

YAML

```yaml
constraints:
  min: 18                       # numbers, dates, and string lengths
  max: 65
  min_length: 40
  max_length: 300
  pattern: "^EMP-\\d{6}$"       # a regular expression the value must match
  enum: [active, suspended]     # the only permitted values
  forbidden: [test, TODO]       # values that must not appear
  multiple_of: 5
  precision: 2                  # decimal places
```

`min` and `max` work on numbers, on dates (written as ISO strings) and on string lengths. Constraints are enforced after generation, and they are also read *backwards*: several generators consult them so that what they produce is valid in the

first place, and the edge-case injector uses them to find the boundaries worth testing.

8.5 Nullability

```yaml
  manager:
    nullable: true              # 5% null
  discount_code:
    null_probability: 0.85      # 85% null, explicitly
```

Nulls introduced this way are part of the design and are not damage: the validator expects them and the linter will tell you, at `info`, when half a column or more is null in case that was a stray decimal point.

8.6 Dependencies and ordering

Field order in the output is the order you wrote them. *Generation* order is computed. A field is generated after everything it depends on, and dependencies come from three places:

1. What the generator declares — `reference` depends on its target entity, `tts` on its `source` field.
2. What a template or expression mentions — `"{first_name|lower}.{last_name|lower}@example.com"` creates two dependencies.
3. `depends_on:`, for the cases the first two cannot see.

A cycle is a compile error that names the cycle. `cacophony plan` prints the resolved order with the reason beside each field.

CHAPTER

9

Relationships, and what really makes one

THERE IS A `RELATIONSHIPS:` BLOCK, AND IT IS DOCUMENTATION. WHAT ACTUALLY CREATES A foreign key is a field with `generator: reference`.

```yaml
relationships:
  - from: company
    to: employee
    cardinality: one_to_many    # one_to_one, one_to_many, many_to_one,
many_to_many
    field: company_id
    required: true
```

Declaring a relationship affects entity ordering and is carried into the plan and the Studio's graph view, which is a real benefit when a reader is trying to understand a large schema. It creates nothing. A project with references and no `relationships:` block generates identical data; a project with the block and no reference fields generates unrelated entities that merely claim to be related.

CHAPTER

10

Outputs

Where records go and in what shape. The command line names one format; the `outputs:` block names whole layouts, so a project can declare its analytics shape once and you pick it by name.

```yaml
outputs:
  analytics:
    format: parquet
    path: out/analytics
    partition_by: [region, opened_on]
    options: {compression: zstd}
  fixtures:
    format: csv
    path: out/fixtures
    entities: [employee]
```

```
$ cacophony generate project.yaml --output-profile analytics
$ cacophony generate project.yaml --output-profile fixtures
```

A profile is a named set of the options `generate` would otherwise take on the command line. Naming one selects it; an explicit flag still wins, so `--output-profile analytics -d /tmp/scratch` writes the analytics layout somewhere else. Two profiles means running the command twice — which is also what "the same logical dataset, written two ways" means in practice.

A profile's relative `path:` resolves against the **schema file**, like every other path in a project. `--out-dir` is a command-line argument and stays relative to where you are standing.

Output formats

Format	Writes	Notes
`jsonl`	One JSON object per line, one file per entity	The default. Appendable, streamable, resumable in place.
`ndjson`	Identical to `jsonl`	An alias, for tools that expect the name.

FORMAT	WRITES	NOTES
`json`	One JSON array per entity	Cannot be appended, so a resume opens a new part file.
`csv`	One file per entity, header row included	Nested values are JSON-encoded in the cell.
`parquet`	Columnar, optionally partitioned	Needs `pip install 'cacophony[parquet]'`. Resume opens a new part.
`sqlite`	One database for the whole project	Every entity a table, every reference an enforced `FOREIGN KEY`.
`sql`	A portable `CREATE TABLE` plus `INSERT` script per entity	For a database Cacophony has no adapter for.

```
TERMINAL
$ cacophony generate project.yaml -o sqlite -d out/
$ sqlite3 out/retail-commerce.db "PRAGMA foreign_key_check"   # silent
```

10.1 Partitioning

`partition_by` turns column values into directories, which is the layout every columnar reader already understands:

```
out/analytics/employee/region=emea/opened_on=2026-03-01/employee.parquet
```

One file per distinct combination, opened when the first record needing it arrives. Partitioning on a high-cardinality column is how one dataset becomes a million tiny files, so a run stops at 512 open partitions and says so; `options: {max_partitions: N}` raises it. A null key becomes `__null__`, because `region=/` is not a path.

Whether the partition columns *also* stay inside the files is decided by what the format's readers do with them, which is not a matter of taste:

- **Parquet** reconstructs them from the directory names, and refuses a directory where they appear in both — *"Field region has incompatible types: string vs dictionary"* — so they are dropped from the files.
- **JSON Lines, CSV and JSON** have no such convention. Nothing would put the column back, so it is kept.

`options: {drop_partition_columns: false}` overrides either way. `sqlite` and `sql` cannot be partitioned and say so: there is one destination for the whole project, so there is nothing to divide.

A PARTITIONED RUN RESUMES INTO NEW PARTS

A partitioned writer is not appendable — working out how far through *each* partition a killed run got is a great deal of bookkeeping for no benefit. A resume therefore writes new part files inside each partition, which every reader for these formats accepts.

CHAOS AND A DATABASE SCHEMA CANNOT BOTH BE HAD

Entropy injection nulls required fields, mangles keys and re-emits whole records the way a retried insert does. Every one of those a declared constraint would reject, aborting the run on its first damaged row. A run with `chaos` enabled therefore writes its tables *without* primary keys, uniqueness, `NOT NULL` or `FOREIGN KEY`, creates indexes on the key and reference columns instead so the joins still work, and says so when it starts.

Generate without chaos for a database whose constraints hold; generate with it for a corrupted one to test a loader against. Asking for both silently would give you neither.

PART IV

Generators

Thirty strategies for producing a value. Twenty-six are deterministic, four need a server, and one is deliberately not implemented.

CHAPTER

11

Choosing a generator

You do not have to choose. A field with a clear `semantic` and a type gets a generator recommended for it, and the recommendation is visible in `cacophony plan` before anything is generated. But the choice is usually obvious once the catalogue is laid out, and naming one is how a schema documents its intent.

11.1 The registry

TERMINAL

```
$ cacophony generators
$ cacophony generators --json | jq '.[] | select(.cost_class=="llm")'
```

ALL THIRTY GENERATORS

NAME	ALIASES	COST	DETERMINISTIC	PRODUCES
constant	fixed, literal	cpu	yes	The same value, always
sequence	serial, autoincrement	cpu	yes	Numbered identifiers from the record index
uuid	guid	cpu	yes	A UUID, stable for a given seed
random	rand	cpu	yes	Random values within the field's constraints
boolean	bool, flag	cpu	yes	A weighted coin flip
weighted	choice, categorical, enum	cpu	yes	A choice from a weighted set
distribution	dist, statistical	cpu	yes	A draw from a statistical distribution
lookup	dataset, from_list	cpu	yes	A value from a list or a data file
pattern	shape, mask	cpu	yes	Template-based identifiers
template	interpolate, format	cpu	yes	Other fields interpolated into a string
expression	expr, derived, computed	cpu	yes	A value computed from other values
composite	chain, pipeline	cpu	yes	Generators in sequence

NAME	ALIASES	COST	DETERMINISTIC	PRODUCES
transform	post_process	cpu	yes	Operations applied to a value
null	none, empty	cpu	yes	Null, always
datetime	date, time, timestamp, temporal	cpu	yes	A moment drawn from a window
ip	ip_address, ipv4	cpu	yes	An IP address
mac	mac_address	cpu	yes	A hardware address
phone	phone_number, telephone	cpu	yes	A telephone number
government_id	ssn, national_id	cpu	yes	An identifier that cannot belong to anyone
faker	fake	cpu	yes	Any Faker provider, made safe
reference	fk, foreign_key, belongs_to	cpu	yes	A foreign key that resolves
event_time	occurred_at, timeline	cpu	yes	When this event happened
subject	actor, belongs_to_subject	cpu	yes	Whose event this is
state	running, accumulated	cpu	yes	A value folded over earlier events
scenario	incident, affected_by	cpu	yes	What a scenario is doing to this record
llm	language_model, ai, gpt	llm	no	Prose, from a language model
image	invokeai, text_to_image	gpu	no	A picture
tts	speech, voice	gpu	no	Audio of generated text
document	pdf, invoice, report	cpu	no	A rendered document
script	custom, python	cpu	—	Declared, and deliberately not implemented

11.2 Cost classes

Every generator declares what it costs, and the number is used rather than displayed. `cpu` work is generated in parallel across entities; `llm` work is batched and rate-limited by the provider's declared concurrency; `gpu` work is usually serialised because there is one graphics card. The distributed controller reads a shard's requirements from exactly this information, so a node with no model server is never handed model work.

11.3 Recommendation

Omit `generator:` and one is chosen from the field's name, its `semantic`, its type and its constraints. An `email` type gets a safe address; a `decimal` with a

`precision` constraint gets a distribution; a field whose semantic is a sentence of English about what a human would write gets `llm` if a model provider is configured and a marked placeholder if not.

> **RECOMMENDATION IS A STARTING POINT, NOT MAGIC**
>
> Run `cacophony plan` and read what you were given. The recommender is good at the obvious cases and has no opinion about your domain: it cannot know that your `status` field should be 70% resolved, or that your salaries are lognormal. Naming the generator is how you tell it.

CHAPTER

12

Values from nothing

Generators that need no other field, no server and no context beyond the record's position. These are the backbone of every schema.

12.1 constant

```
generator: constant  ·  aliases: fixed, literal
```

value

ANY The value to produce. Required.

```yaml
source_system:
  type: string
  generator: constant
  value: "hr-core"
```

12.2 sequence

```
generator: sequence  ·  aliases: serial, autoincrement
```

format

STRING A pattern with a digit group, e.g. `"USER-{000000}"`. The number of zeros is the zero-padded width.

start

INTEGER First value. Default `1`.

step

INTEGER Increment. Default `1`.

pad

INTEGER Zero-padding width, when no `format` is given.

```yaml
employee_id:
  generator: sequence
  format: "EMP-{000000}"     # EMP-000001, EMP-000002, …
```

Derived from the record index rather than a counter, so record 4,823,913 has the same identifier however it is reached — in a resumed run, in a shard on another machine, or on its own from `regenerate`.

12.3 uuid

```
generator: uuid  ·  alias: guid
```

version

`4 | 5` Default `4`.

namespace

`UUID` For version 5.

name

`STRING` The name hashed for version 5; may be a `{field}` template.

string

`BOOLEAN` Emit as a string rather than a UUID object. Default true for most writers.

Version 4 draws its bits from the field's derived seed, so it looks random and is entirely reproducible. Version 5 is a hash of a namespace and a name, which is how you make a UUID that is a function of the record's own data.

12.4 random

```
generator: random  ·  alias: rand
```

min, max

`NUMBER` Bounds, for numeric fields. Aliases `low`, `high`, `minimum`, `maximum`.

precision

`INTEGER` Decimal places.

length, min_length, max_length

`INTEGER` For strings.

charset

`STRING` `alpha`, `alphanumeric`, `lower`, `upper`, `digits`, `hex`, or a literal set of characters.

Behaviour follows the field's declared type: integers get integers, strings get characters from the charset, and temporal fields hand off to `datetime`.

12.5 boolean

```
generator: boolean  ·  aliases: bool, flag
```

probability

`0.0–1.0` Chance of `true`. Default `0.5`. Alias `true_probability`.

12.6 weighted

```
generator: weighted  ·  aliases: choice, categorical, enum
```

choices

`MAPPING | LIST` A mapping of value to weight, a plain list (equal weights), or a list of `{value, weight}`. Aliases `values`, `items`, `options`.

YAML

```yaml
operating_system:
  type: enum
  generator: weighted
  choices:
    Windows 11: 52
    Windows 10: 15
    macOS 15: 18
    Ubuntu 24.04: 13
    Other: 2
```

Weights need not sum to anything in particular; they are normalised. This is the generator that does most of the work of making a dataset look real, because almost nothing in the world is uniformly distributed.

12.7 distribution

```
generator: distribution  ·  aliases: dist, statistical
```

distribution

`ENUM` `uniform`, `normal`, `lognormal`, `exponential`, `poisson`, `beta`, `histogram`.

mean, stddev

`NUMBER` For `normal` and `lognormal`. Aliases `mu`, `sigma`, `std`.

`rate, scale`

NUMBER For `exponential`.

`lam`

NUMBER For `poisson`. Alias `lambda`.

`alpha, beta`

NUMBER For `beta`. Aliases `a`, `b`.

`bins`

LIST For `histogram`: bucket edges and weights.

`min, max`

NUMBER Clamp the result. Not a re-draw — the tail piles up on the bound, which is usually what you want for an age or a salary.

`precision`

INTEGER Decimal places.

YAML

```yaml
cvss_score:
  type: float
  generator: distribution
  distribution: beta
  alpha: 2.4
  beta: 2.0
  min: 0.1
  max: 10.0
  precision: 1
```

> **LOGNORMAL PARAMETERS ARE IN LOG SPACE**
>
> A `lognormal` with `mean: 11.2` has a median of $e^{11.2}$ — about 73,000, which is a salary. Writing `mean: 73000` produces numbers with more digits than money has. The linter mentions this at `info` when it sees a suspiciously large log-space mean.

12.8 lookup

`generator: lookup` · `aliases: dataset, from_list`

`values`

LIST Inline values. Aliases `choices`, `items`, `list`.

`path`

PATH A `.csv`, `.json` or `.txt` file. Relative to the *schema file*. Aliases `file`, `source`.

`column`

STRING Which column of a CSV. Alias `key`, `field`.

mode

ENUM `random` (default) or `cycle`.

This is how a project uses real reference data — a list of product categories, a table of airport codes — without inventing it. The file travels with the project in a bundle, because export follows the paths in the expanded project.

12.9 null

```
generator: null  ·  aliases: none, empty
```

Always produces null. Useful for a placeholder column that must exist in the output shape, and for negative tests.

CHAPTER

13

Shapes, text and derivation

Five generators that build a value out of structure or out of other fields. Between them they cover most of what people reach for a scripting language to do — which is the point, since Cacophony will not run one.

13.1 pattern

`generator: pattern` · `aliases: shape, mask`

pattern

STRING Required. The shape, with token groups in braces.

upper, lower

BOOLEAN Force the case of the result.

PATTERN TOKENS

TOKEN	PRODUCES
{A-Z}	One character from the range
{A-Z:3}	Three characters from the range
{0000}	Four random digits
{???}	Three random letters
{***}	Three random alphanumerics
{{ }}	A literal brace

YAML

```yaml
server_tag:
  generator: pattern
  pattern: "SRV-{A-Z}{A-Z}-{0000}"      # SRV-KQ-4817
```

A NUMERIC RANGE IS NOT A TOKEN

`{10000-99999}` looks like it should produce a number in that range and does not — the range syntax is for *characters*. Use `{00000}` for five digits, or the `random` generator with `min` and `max` for a genuine numeric range. Three recipes in the catalogue got this wrong before the catalogue was tested per recipe.

13.2 template

`generator: template` · `aliases: interpolate, format`

template

STRING Required. Text with `{field}` placeholders.

on_missing

ENUM `empty` (default), `error`, `keep`.

Placeholders name sibling fields, or `{related.field}` to read through a reference. Filters chain with `|`:

TEMPLATE FILTERS

FILTER	EFFECT
`lower` · `upper` · `title`	Case
`strip` · `nospace`	Trim, or remove all spaces
`slug`	Lowercase, hyphenated, ASCII
`initial`	First character
`trunc:N`	At most N characters
`pad:N`	Zero-pad to width N

YAML

```yaml
email:
  type: email
  generator: template
  template: "{first_name|lower}.{last_name|lower}@example.com"
badge:
  generator: template
  template: "{first_name|initial}{last_name|initial}-{employee_id|trunc:6}"
```

13.3 expression

`generator: expression` · `aliases: expr, derived, computed`

expression

STRING Required. Aliases `expr`, `formula`.

Evaluated from a parsed syntax tree with an allow-list of node types and functions. There is no `eval`, no imports, and no attribute access beyond dotted record lookups — because a project file is something people send each other.

EXPRESSION FUNCTIONS

GROUP	FUNCTIONS
Text	`lower upper title strip len str slug concat join replace substr`
Numbers	`int float round abs min max sum`
Logic	`bool coalesce iif when`
Dates	`year month day format_date`
Tests	`contains startswith`
Hashing	`hash`

```yaml
username:
  generator: expression
  expression: 'lower(substr(first_name, 0, 1) + last_name)'
unit:
  generator: expression
  expression: >
    iif(metric == 'temperature', 'celsius',
      iif(metric == 'humidity', 'percent', 'ppm'))
```

It is `iif(condition, a, b)` rather than `if`, because `if` is a Python keyword and the parser is Python's own.

13.4 composite

`generator: composite` · `aliases: chain, pipeline`

steps

`LIST` Generators to run in order, threading the value through. Aliases `generators`, `pipeline`, `ops`.

13.5 transform

`generator: transform` · `alias: post_process`

operations

`LIST | STRING` Operations to apply, joined with `|` if written as a string.

source

`STRING` Which field to read. Defaults to this field's own generated value. Alias `from`, `field`.

```yaml
  email_hash:
    generator: transform
    source: email
    operations: "lowercase|hash:16"
```

The operations are the same set available in `patches:` and on the command line, listed in full in appendix C. Every one is deterministic, including `add_noise`, whose jitter is derived by hashing the value rather than drawn from a generator.

CHAPTER

14

Identifiers of the real world

Six generators produce the values that look most like personal data, and all six are constrained by default to ranges that cannot belong to anyone.

14.1 datetime

```
generator: datetime  ·  aliases: date, time, timestamp, temporal
```

start, end

ISO DATE OR DATETIME The window to draw from.

business_hours

BOOLEAN Restrict to working hours. Aliases `office_hours`.

weekdays_only

BOOLEAN Skip weekends. Alias `business_days`.

timezone_offset

NUMBER Hours from UTC. Alias `tz_offset`.

Returns whatever the field declares: a `date` field gets a date, a `time` field a time, a `datetime` field both. For events that should follow a project-wide shape rather than a flat window, use `event_time` and a `timeline:`.

14.2 ip

```
generator: ip  ·  aliases: ip_address, ipv4
```

network

CIDR Draw from this network. Aliases `cidr`, `subnet`.

version

4 | 6 Default 4.

safe

BOOLEAN Default true: RFC 5737 and RFC 3849 documentation ranges.

Naming a `network` takes precedence, which is how you generate addresses inside `10.0.0.0/8` for a network your test is about. RFC 1918 private space is not "safe" in the documentation sense but is harmless in practice; the setting exists so the choice is yours and explicit.

14.3 mac

```
generator: mac  ·  alias: mac_address
```

oui

STRING A specific vendor prefix. Aliases `prefix`, `vendor`.

separator

STRING `:` (default) or `-`.

upper

BOOLEAN Uppercase hex.

safe

BOOLEAN Default true: RFC 7042's documentation OUI.

14.4 phone

```
generator: phone  ·  aliases: phone_number, telephone
```

format

ENUM `e164`, `national`, `plain`.

area_code

STRING A specific area code.

safe

BOOLEAN Default true: the 555-0100 to 555-0199 fiction block.

14.5 government_id

```
generator: government_id  ·  aliases: ssn, national_id
```

masked

BOOLEAN Emit as `***-**-1234`.

safe

BOOLEAN Default true: the never-issued 900 area.

14.6 faker

```
generator: faker  ·  alias: fake
```

provider

STRING The Faker provider to call, e.g. `company`, `job`, `street_address`. Aliases `method`, `faker`.

locale

STRING Overrides the project locale.

unique

BOOLEAN Ask Faker for distinct values.

safe

BOOLEAN Default true. Domain-bearing values are rewritten onto reserved domains.

Any option Cacophony does not recognise is passed to the provider, so `provider: pystr` with `min_chars: 8` works as it would directly. Faker is seeded from the field's derived seed, so it is as reproducible as everything else here.

CHAPTER

15

Language models

One generator, `llm`, for the fields that are genuinely prose. It has no `prompt` option, and that absence is the design.

15.1 No prompt

A field already says what it means (`semantic`), what type it is, what length it must be (`constraints`), what voice to use (`tone`), what good answers look like (`examples`) and what else about the record matters (`context`). All of that is already in the schema, so the prompt compiler assembles the prompt section 12. Writing one by hand would mean saying the same things twice and having them disagree.

```
TERMINAL
$ cacophony prompt project.yaml -e ticket --schema
```

That command prints exactly what will be sent, including the JSON Schema used to constrain decoding. It is the answer to "what is it actually asking".

15.2 Options

```
generator: llm  ·  aliases: language_model, ai, gpt
```

provider

STRING Which configured provider to use. Alias `provider_id`.

mode

ENUM `per_field`, `per_record` (default), `batch`, `expansion`.

context

LIST Which already-generated fields the model is shown. Alias `inputs`.

max_tokens

INTEGER Ceiling on the answer.

GENERATORS

temperature

NUMBER Sampling temperature.

on_unavailable

ENUM `error` (default), `placeholder`, `null`.

YAML

```yaml
resolution_notes:
  type: text
  semantic: >
    Natural-language notes written by an IT support technician explaining
    how the ticket was resolved.
  tone: Concise internal enterprise IT helpdesk writing
  generator: llm
  provider: local_llm
  mode: per_record
  context: [category, device_type, status]
  constraints:
    min_length: 40
    max_length: 300
```

15.3 Modes, and what they cost

CALLS FOR 1,000 RECORDS WITH THREE MODEL-WRITTEN FIELDS

MODE	CALLS	WHEN TO USE IT
`per_field`	3,000	Maximum control; each field gets the model's full attention.
`per_record`	1,000	The default. Fields of one record stay mutually consistent.
`batch`	1,000 ÷ `--llm-batch-size`	Bulk generation, when volume matters more than nuance.
`expansion`	1,000	An explicit name for what every mode already does.

15.4 Context

Omit `context` and every deterministic field of the record is offered. That default is deliberate: a model given no context writes a biography that contradicts the record it belongs to, and a dataset where two columns disagree is a broken fixture rather than a realistic one. Name the fields explicitly when the record is wide and only some of it is relevant.

15.5 The model is never trusted

What comes back is extracted from whatever wrapper it arrived in, parsed, type-checked, constraint-checked and repaired where repair needs no judge-

ment — trailing prose after a JSON object, a number quoted as a string, a missing field with an obvious default. What cannot be repaired is retried with a prompt quoting what was wrong, then with the schema restated. The ladder is three model calls long and then it stops sections 13, 66.

Where the provider supports it, decoding is constrained by a JSON Schema so the answer cannot be malformed in the first place: Ollama enforces the schema natively, llama.cpp through a GBNF grammar, and OpenAI-compatible servers by negotiation.

> **CONSTRAINED DECODING CLIPS**
>
> A server enforcing `maxLength` during decoding stops mid-word rather than producing an over-length value. The result passes every check in the platform and is not a sentence: `"…failed to handle queueing mechanism, led 3"`. This is why `cacophony benchmark` has a `CLIPPED` column. If you see it, either the model needs more room or your limit is wrong.

15.6 When there is no model

`on_unavailable: placeholder` produces a marked stand-in fitted to the field's length constraints, so a project with model-written fields still runs end to end on a laptop with no server. Two recipes in the built-in catalogue use exactly this, so that the catalogue works everywhere.

> **FITTED, AND IT HAD TO BE MADE SO**
>
> There were two paths to a placeholder — one field at a time, and a whole enrichment group — and only one of them remembered to clamp the result. The shipped helpdesk template, whose subject line is capped at ninety characters, was getting a hundred-and-nine-character stand-in. It was invisible while invalid records were written anyway; the first run under an aborting validator found it immediately. The clamp now lives where the value is made, so no caller can forget it.

CHAPTER

16

Media

Three generators write files: pictures, speech and documents. Each puts a path in the record and the bytes under `assets/`, with a manifest tying the two together.

16.1 image

`generator: image` · `aliases: invokeai, text_to_image`

prompt

STRING A `{field}` template over this record. Without one, the field's semantic plus the record's values stand in.

style

ENUM For the procedural adapter: `identicon`, `portrait`, `card`, `document`.

width, height

INTEGER Pixels.

steps, guidance

NUMBER Diffusion parameters. Alias `cfg_scale`.

negative_prompt

STRING Alias `negative`.

workflow

PATH An exported InvokeAI graph, for architectures the built-in graph does not cover.

provider, reuse, on_unavailable

– As elsewhere.

```yaml
  portrait:
    type: image
    generator: image
    prompt: "corporate headshot of {first_name} {last_name}, {team} agent"
    width: 256
    height: 256
    style: portrait
```

Because the prompt is a template over the record, the compiler orders the image after the fields it names — you cannot accidentally render a portrait before the name exists.

16.2 tts

```
generator: tts  ·  aliases: speech, voice
```

source

STRING Required. The field holding the words.

voice

STRING A voice name the provider knows.

voice_field

STRING Take the voice from a field, so a speaker is consistent per subject. Alias `voice_from`.

speed, language, sample_rate

– Synthesis parameters.

provider, reuse, on_unavailable

– As elsewhere.

The clip's duration and an aligned transcript go into the asset manifest, which is what makes a generated speech set usable for training or testing rather than merely present.

16.3 document

```
generator: document  ·  aliases: pdf, invoice, report
```

template

STRING Inline body text with `{field}` placeholders. One of this or `template_path` is required.

`template_path`

: PATH A file holding the same, relative to the schema.

`format`

: ENUM `pdf`, `html`, `txt`.

`title`

: STRING Also a template.

`page_size`

: STRING `a4`, `a5`, `letter`.

`font, font_size`

: — Typeface and size.

```yaml
id_badge:
  type: file
  generator: document
  format: pdf
  page_size: a5
  title: "{first_name} {last_name} - {employee_id}"
  template: |
    {first_name} {last_name}
    {employee_id}
    Team: {team}
```

No provider is needed: a document is rendered in-process from the record it describes. Templates take `{field}` and `{related.field}` and nothing else — deliberately not a template *language*, since a project file is something people share.

16.4 Where the files go

```
out/
├── employee.jsonl
└── assets/
    ├── manifest.jsonl
    └── employee/00000000/employee_00000000_portrait.png
```

Paths are derived from entity, record index and field, so a resumed run reuses what it already produced rather than paying for it again. Identical bytes are stored once section 81. The manifest records one line per asset:

```json
{"entity": "employee", "record_index": 0, "field": "portrait", "kind": "image",
 "media_type": "image/png", "size_bytes": 2588, "record_id": "SUP-0001",
 "metadata": {"provider": "pictures", "workflow": "procedural:portrait",
              "seed": 1585453150, "prompt_hash": "514e7bbc1af5d2bb"}}
```

`--assets-dir` puts them elsewhere, which is how several machines share one directory; `--regenerate-assets` redraws what is already there.

16.5 Designing multimodal schemas without a GPU

The `procedural_image` and `procedural_speech` adapters draw and synthesise in-process, deterministically, with no server and no graphics card. Every result is labelled `synthetic: true`. They exist so a multimodal schema can be designed, previewed, tested and demonstrated anywhere; swapping in `invokeai` or `piper` is one line in `providers:`.

PART V

Relations and time

References that resolve and cost nothing, events that fall where events fall, subjects with histories, incidents buried in them, and damage on purpose.

CHAPTER

17

References

A reference is a foreign key that resolves. It is also, in Cacophony, arithmetic rather than bookkeeping — which is why a hundred million events pointing at five thousand employees costs the same memory as five.

17.1 Declaring one

```
generator: reference  ·  aliases: fk, foreign_key, belongs_to
```

entity

STRING Required. Which entity to point at. Aliases `to`, `references`.

field

STRING Which column of the target. Defaults to its primary key. Alias `key`.

distribution

ENUM `uniform` (default), `skewed`, `sequential`, `round_robin`.

skew

NUMBER How steep `skewed` is. Default `1.6`.

unique

BOOLEAN One child per parent. Forces `sequential`.

expose

BOOLEAN Make the parent's other fields readable by templates and expressions.

on_unavailable

ENUM What to do when the target has no records.

```yaml
order:
  count: 45000
  fields:
    customer:
      generator: reference
      entity: customer
      distribution: skewed
      skew: 1.9
```

A reference with no declared `type` takes the type of the key it points at, because an integer key referenced by a string column joins to nothing.

17.2 How it works

Pick a parent index from this record's derived seed, then generate the transitive closure of that parent's key field — that is, produce just enough of the parent to know its identifier. Nothing is held in a pool and nothing is looked up. The consequence is that references are free at any scale and work identically in a shard on another machine, where no parent has ever been generated.

17.3 Distribution is the option that decides whether the data resembles anything

REFERENCE DISTRIBUTIONS

VALUE	BEHAVIOUR	USE FOR
`uniform`	Every parent equally likely	Almost nothing in real life. The default because it is the least surprising.
`skewed`	A power law: the head of the range attracts most references	Support tickets, orders, logins, API calls — anything people do.
`sequential`	Parent `record_index % count`	Guaranteeing every parent appears.
`round_robin`	The same thing under its other name	—

WHAT `skew` DOES: SHARE OF REFERENCES TAKEN BY THE BUSIEST TENTH OF PARENTS

SKEW	TOP 10% TAKE	LOOKS LIKE
`1.0`	10%	Uniform
`1.6`	24%	The default: mild realism
`2.0`	32%	A busy support queue
`3.3`	50%	Half the traffic from a tenth of users
`7.2`	80%	The Pareto cliché, and some systems really do this

17.4 Reading the parent's other fields

```yaml
employee:
  fields:
    employer:
      generator: reference
      entity: company
    email:
      generator: template
      template: "{first_name|lower}@{company.domain}"    # this row's company
```

The compiler orders `email` after `employer` on its own. Where a field and the entity it points at share a name — a `customer:` field holding a reference to the `customer` entity — a dotted read means the related record, not the key.

17.5 Self-references

```yaml
  manager:
    generator: reference
    entity: $self          # in a recipe; write the entity's own name directly
    nullable: true
```

A self-reference points **backwards**: record *n* may only point at a record before it, and record 0 gets null. That is not a stylistic choice. A forward-pointing self-reference makes the graph cyclic, which means no valid insertion order exists, which means an enforced foreign key in the SQLite output fails on the first row. Backwards-only gives you a real management hierarchy that loads into a real database.

17.6 Uniqueness

`unique: true` gives each parent exactly one child and forces `sequential`, which is how you model a one-to-one extension table. The linter refuses the arithmetic impossibility — more children than parents with `unique` set is `unique-reference-overflow`, an error, before you spend an hour generating.

CHAPTER

18

Timelines and simulation

A dataset of events wants two things a flat random window cannot give it: events that fall when events actually fall, and subjects whose histories accumulate. `timeline:` handles the first, `simulation:` the second.

18.1 The project timeline

```yaml
timeline:
  start: "2026-01-01"
  end: "2026-04-01"
  shape: business_hours       # flat, business_hours, office, retail, evening
  holidays: ["2026-01-01", "2026-02-16"]
  holiday_weight: 0.05        # 0.0 silences a holiday entirely
  months: {december: 2.5}     # seasonality, by name or number
  spikes:
    - {start: "2026-03-01", end: "2026-03-07", multiplier: 8}
  growth: 1.4                 # activity at the end relative to the start
```

TIMELINE KEYS

KEY	MEANING
`start, end`	The window. ISO dates or datetimes.
`shape`	The daily and weekly rhythm: `flat`, `business_hours`, `office`, `retail`, `evening`.
`holidays`	Dates whose weight is multiplied by `holiday_weight`.
`holiday_weight`	Default is a small fraction; `0.0` silences the day completely.
`months`	Seasonal multipliers, by name or number.
`spikes`	Windows with a multiplier: a campaign, an outage, a sale.
`growth`	End-to-start activity ratio, applied smoothly across the window.

18.2 How events land, and why they are already sorted

The shape is compiled into a cumulative distribution function once. Drawing the k-th of n events at quantile k/n and inverting the CDF yields events that are already in chronological order section 25.

No sort. That is not a micro-optimisation: sorting is the step that makes a streaming generator hold the whole dataset, and avoiding it is why a hundred million timestamped events cost the same memory as a hundred.

18.3 Simulation

```yaml
transaction:
  count: 500000
  simulation:
    subject: account          # whose events these are
    distribution: skewed      # uniform, skewed, zipf
    skew: 1.9
    minimum: 4                # events every subject gets before the rest
    state:
      balance:
        initial: "500"
        update: "balance + amount"
        min: 0
        precision: 2
```

Declaring `simulation:` turns an entity's records into a history. Three generators become available and are usually all you need:

GENERATOR	PRODUCES
`subject`	Whose event this is. The allocation already decided; this field reports it. `field:` chooses which column of the subject to point at.
`event_time`	When it happened, drawn from the timeline. `jitter`, `offset`, `offset_field`, `spread` (`ordered` or `random`).
`state`	A value folded over this subject's earlier events. `variable` names which one; it defaults to the field's own name.

18.4 Partitioned folds

Events are laid out in contiguous blocks per subject section 26. Account 41's events are records 912,000 to 912,338, in order. A running balance is therefore a linear fold over one block, needing no lookahead and no global state, and a resumed run replays one subject's block rather than the dataset.

```
records:  0 ──────────── 12 ──────── 27 ──────────────── 61 ─── …
subject:  ├── account 1 ──┤├ account 2 ┤├──── account 3 ─────┤
state:    500 → 480 → 512   500 → 466    500 → 690 → 655 → 700
timeline: each block spread across the project window by quantile,
          so a subject's events are chronological without a sort
```

Contiguous blocks are what make a stateful simulation streamable.

State expressions use the same restricted evaluator as `expression`: no imports, no attribute access, no way to run code out of a shared project file. `min`, `max` and `precision` clamp and round the accumulated value, which is how a balance stays a two-decimal number that never goes negative.

> **STREAMING IS DIFFERENT, AND SAYS SO**
>
> In a live stream, subjects interleave — that is what a stream *is* — so per-subject state is held in memory rather than folded over a contiguous block. The values are the same kind of thing; they are not the same values as a batch run, and nothing pretends otherwise.

CHAPTER

19

Scenarios

A scenario is a behavioural overlay: something that happens to a fraction of subjects, over a window of their history, changing what their events look like. It is how you bury an incident in a year of logs without writing the incident by hand.

YAML

```yaml
scenarios:
  ransomware:
    description: Access, then execution, then encryption.
    applies_to: [authentication]
    affects_fraction: 0.004
    parameters:
      subject: user
      window: {at: 0.62, duration: 0.12}    # or [start, end]
      rate_multiplier: 4.0
      effects:
        risk_score: 88
      phases:
        - name: initial_access
          effects:
            result: {success: 70, failure_bad_password: 30}
        - name: encryption
          effects:
            application: Admin Console
            risk_score: 99
```

SCENARIO KEYS

KEY	MEANING
`applies_to`	Which entities the overlay touches.
`affects_fraction`	What proportion of subjects are involved.
`subject`	Which subject dimension is affected.
`window`	A fraction of the subject's own history: `{at, duration}` or `[start, end]`.
`rate_multiplier`	How much busier the subject gets inside the window.
`effects`	Field values to impose. A mapping is a weighted choice; a string beginning `=` is an expression over the record.
`phases`	Ordered stages within the window, each with its own effects.

19.1 Who is affected, and why no list is held

Subjects are selected by hashing the project seed with the scenario's name. The same identities are affected on every run, in any order, at any scale, and no list of victims is ever materialised. Ask for four in a thousand out of ten million subjects and Cacophony does not build a set of forty thousand identifiers; it asks, per subject, whether this one is in.

19.2 Windows are the same calendar moment for everyone

`window` is a fraction of the subject's history rather than a date. Because events are laid out by timeline quantile, 0.62 of the way through one subject's history is the same calendar moment as 0.62 of the way through another's — however many events each of them has. The incident happens across the population at once, which is what an incident does.

19.3 Reporting it

`generator: scenario` · `aliases: incident, affected_by`

report

ENUM `name` (default), `phase`, `involved`, `position`.

normal

ANY The value for a record no scenario touches.

```yaml
  scenario:
    type: string
    generator: scenario
    report: name
    normal: null
    nullable: true      # the label is absent for most records, and that is
fine
```

A scenario field is the ground truth column: it says which records belong to the incident, which is exactly what you need to score a detection system and exactly what you must remember to remove before handing the data to one.

> **MAKE THE LABEL NULLABLE**
>
> Most records are not in any scenario, so a non-nullable `scenario` field fails validation on ninety-nine per cent of the dataset. Write `nullable: true`, or give `normal:` a value like `"none"`.

CHAPTER

20

Chaos

Deliberate damage. Data your schema forbids, injected on purpose, so you can find out what your pipeline does with input that is wrong rather than merely unusual.

```yaml
chaos:
  preset: realistic       # pristine, realistic, messy, hostile_qa, absolute
  missing_data: 0.05      # anything stated explicitly overrides the preset
```

KINDS OF DAMAGE

KIND	WHAT IT DOES
missing_data	Nulls a field that had a value
malformed_text	Whitespace, case, transpositions, truncation, stray tabs
unexpected_unicode	Zero-width spaces, RTL marks, combining accents, emoji
outliers	Numbers orders of magnitude out, sometimes negative
temporal_anomalies	Dates in the far future or past, or in another format entirely
referential_anomalies	A well-formed key that points at nothing
duplicates	Emits the record twice, the way a retried insert does

PRESETS, AS FRACTIONS OF RECORDS AFFECTED

KIND	PRISTINE	REALISTIC	MESSY	HOSTILE_QA	ABSOLUTE
missing_data	—	0.02	0.06	0.10	0.20
malformed_text	—	0.004	0.03	0.08	0.20
unexpected_unicode	—	0.001	0.01	0.05	0.15
outliers	—	0.005	0.02	0.05	0.15
temporal_anomalies	—	0.0005	0.005	0.02	0.08
referential_anomalies	—	—	0.002	0.01	0.05
duplicates	—	0.001	0.01	0.02	0.05

20.1 The shortcut

`profile: maximum_chaos` in the `project:` block turns on the `hostile_qa` preset and five per cent edge cases in one line, for when the point of the dataset is to be difficult. A `chaos:` block still overrides it.

20.2 What chaos never touches

Primary keys and `scenario` fields. A damaged primary key is not a robustness test; it is a dataset whose joins have stopped resolving, after which every finding is about Cacophony rather than about your pipeline.

20.3 Damage is recorded

Every damaged field is noted in `_chaos` on the record and in provenance, and the validator skips those fields — otherwise a chaotic run is a wall of validation failures reporting the feature working, and `--drop-invalid` discards precisely what you asked for.

20.4 Chaos against a database output

See the warning at the end of chapter 10: a run with chaos writes its tables without constraints and with indexes instead, because damaged data and enforced integrity are mutually exclusive by construction. This is stated when the run starts rather than discovered when it aborts.

PART VI

Reuse

Fragments you do not write twice, populations that persist between datasets, projects that can be sent to somebody else, and eight complete examples to steal from.

CHAPTER

21

Recipes

A recipe is a named group of fields. One line on an entity expands into the twelve fields you were about to type, and naming one of them again overrides it without restating the rest.

YAML

```yaml
entities:
  employee:
    count: 5000
    recipes: [employee]        # twelve fields, one line
    fields:
      email:                   # override one without restating the rest
        template: "{first_name|lower}.{last_name|lower}@acme.example"
      cost_centre:             # and add your own
        type: string
        generator: pattern
        pattern: "CC-{0000}"
```

TERMINAL

```
$ cacophony recipes                    # every recipe, by group
$ cacophony recipes --show employee    # its fields, and where each came from
$ cacophony recipes --group security --json
```

21.1 Expansion is visible

Every expanded field records the recipe it came from. `cacophony plan` prints `via employee` beside it and the Studio badges it in the field editor. A schema that silently gained eight fields would be a schema nobody can debug.

21.2 Overriding does not require forking

Naming a field the recipe defines overrides it key by key, *in the place the recipe put it*, so the record does not reorder. Naming a different `generator` replaces the generator and its options wholesale — merging `generator: llm` onto `generator: template` would leave the template's options behind as junk.

21.3 Where recipes come from

In increasing precedence: the built-in catalogue, a `recipes/` directory beside the project file, then the project's own `recipes:` block. A project may therefore replace a built-in by defining one with the same name.

```yaml
recipes:
  cost_centre:
    description: Our own coding.
    fields:
      cost_centre: {type: string, generator: pattern, pattern: "CC-{0000}"}
```

A recipe may `includes:` others; cycles are refused by name. `$self` is the only substitution — it becomes the name of the entity being expanded into, which is what a manager relationship needs.

21.4 The built-in catalogue

Thirty-one recipes in five groups section 106. Every one compiles, generates and passes its own validation, asserted per recipe in the test suite: a catalogue is a promise, and a recipe that does not work is worse than no recipe. The full listing with fields is appendix B.

THE CATALOGUE AT A GLANCE

GROUP	RECIPES
identity	name · person · employee · customer · username · email · address
computing	hostname · ip · mac · os · browser · device · software_version
security	cvss_score · cve_identifier · alert_severity · logon_event · network_event · hash_value
commerce	product · sku · transaction · invoice · price · currency
operational	ticket · status · priority · comment · timestamp

Everything is deterministic except `comment` and `ticket`, whose prose fields use a language model and carry `on_unavailable: placeholder` so a project with no model server still runs. Section 62's reserved ranges hold throughout.

CHAPTER

22

Worlds

A world is a name for a seed and the schema that goes with it. Generating against one produces the same people every time, so a dataset of logins made today and a dataset of tickets made next week describe the same company.

```
TERMINAL
$ cacophony worlds project.yaml --create acme-2026
$ cacophony worlds project.yaml
$ cacophony worlds project.yaml --show acme-2026
$ cacophony generate project.yaml --world acme-2026 -o jsonl -d out/
$ cacophony worlds project.yaml --delete acme-2026
```

`--show` does more than print: it checks that the schema still matches the one the world was created from. A world whose schema has drifted no longer describes the same population, and being told so is the entire value of recording it.

> **WHY NOT JUST WRITE DOWN THE SEED**
>
> You can, and for a one-off that is enough. A world adds the thing a bare integer cannot: the association with a specific schema, checked. Six months later, "seed 20260816" tells you nothing about whether the file in front of you is the one that produced the dataset in the ticket.

CHAPTER

23

Bundles

A project that can be sent to somebody else: one file holding the schema and everything it references, with an importer that treats the archive as the untrusted input it is.

```
TERMINAL
$ cacophony bundle export project.yaml -o team.cacophony
$ cacophony bundle inspect team.cacophony
$ cacophony bundle import team.cacophony -d ./team
```

23.1 What is in one

A `.cacophony` file is a zip holding `project.yaml`, a manifest with a SHA-256 per entry, and the supporting directories section 72 lists — `recipes/`, `schemas/`, `templates/`, `workflows/`, `scripts/`, `assets/` — plus `worlds/`, since a named world is part of what reproduces a dataset.

23.2 Four rules that make it work

- **Generated data is never included.** Section 72 is explicit, and a project is kilobytes while a dataset is gigabytes. `.jsonl`, `.parquet`, `.db` and friends are excluded wherever they sit.
- **Files the schema references are packed wherever they live.** A lookup table in `data/` is as much part of the project as one in `templates/`, so export follows the paths in the *expanded* project rather than guessing at directory names. The first bundle produced during development went out without its own CSV, which is how this rule got written.
- **A path outside the project directory is refused, not dropped.** It cannot travel, and a bundle that quietly lost its lookup table would import cleanly and fail on its first record.

- **Import treats the archive as hostile.** Path traversal, absolute paths, Windows drive letters and symlink entries are refused; size and entry-count ceilings apply; and the whole archive is checked before a byte is written, so one bad entry leaves nothing behind rather than half a project.

23.3 `inspect` answers "will this work"

It writes nothing. It verifies every file against the manifest's hash, then extracts to a temporary directory and *compiles the whole project* — so you can decide whether to unpack something before unpacking it.

CHAPTER

24

The shipped templates

Eight complete projects in `templates/`. None is a toy: each compiles, generates and validates in the test suite, and each was written to demonstrate a different part of the system.

TEMPLATES/

FILE	DEMONSTRATES
`corporate-directory.yaml`	The basics done well: people, departments, a manager hierarchy that loads into a database.
`helpdesk.yaml`	Tickets, skewed references, model-written prose with a placeholder fallback.
`retail-commerce.yaml`	Products, orders, line items; decimal money; a SQLite output whose foreign keys resolve.
`security-operations.yaml`	Authentication events on a timeline, subjects with state, and a ransomware scenario buried in them.
`saas-application.yaml`	Accounts, subscriptions, usage events; growth over the window; churn.
`iot-telemetry.yaml`	Devices emitting metrics; per-metric units chosen coherently; very high record counts.
`multimodal-support.yaml`	Images, speech and documents together, with the procedural adapters so it runs anywhere.
`conversational-ai.yaml`	Turn-structured dialogue for training or evaluation sets.

TERMINAL

```
$ cacophony plan templates/security-operations.yaml
$ cacophony preview templates/iot-telemetry.yaml -n 5
$ cacophony generate templates/retail-commerce.yaml -o sqlite -d out/
```

READ THEM IN THIS ORDER

`corporate-directory` for the schema language, `retail-commerce` for relational structure, `security-operations` for timelines, simulation and scenarios all at once. The other five are variations once those three make sense.

PART VII

Running it

Every command and every option; where the records go; what a run records about itself; and how to point Cacophony at a model.

CHAPTER

25

The command line

28 commands. This chapter is the complete reference, grouped by what you are trying to do; appendix F has the same material as a one-page synopsis.

25.1 Global behaviour

EXIT CODES

CODE	MEANING
0	Success
1	Lint errors
2	Bad schema or bad arguments
3	Generation failure
4	Run cancelled rather than failed

Errors go to standard error, so `cacophony preview --json | jq` is safe. Most commands taking a project accept `--seed` to override the project seed for that invocation.

25.2 Understanding a schema

25.2.1 `validate`

```
cacophony validate PROJECT [--seed N]
```

Compiles the schema and reports structural problems: unknown generators, bad options, unresolvable dependencies, cycles. Generates nothing.

25.2.2 `lint`

```
cacophony lint PROJECT [--strict]
```

Nineteen rules about schemas that compile and will disappoint you. `--strict` exits non-zero on warnings as well as errors, which is the form to use in CI. Appendix I lists every rule.

25.2.3 `plan`

```
cacophony plan PROJECT [--seed N] [--json]
```

The compiled truth: entities in dependency order, and per field its generator, cost class, determinism, dependencies and recipe attribution.

25.2.4 `prompt`

```
cacophony prompt PROJECT [-e ENTITY] [--batch-size N] [--schema]
```

Prints the prompts the compiler builds, and with `--schema` the JSON Schema used to constrain decoding.

25.2.5 `generators`, `recipes`, `plugins`

```
cacophony generators [--json] cacophony recipes [-p PROJECT] [--show NAME]
[--group NAME] [--json] cacophony plugins [--show NAME] [--json]
```

What is available in this installation: thirty generators, thirty-one recipes plus whatever the project adds, and any plugins installed.

25.3 Producing data

25.3.1 `preview`

```
cacophony preview PROJECT [-e ENTITY] [-n COUNT] [--offset N] [-c COLUMNS]
[--json] [--isolate] [--cache MODE] [--cache-path FILE] [--seed N]
```

Shows exactly the records a real run would produce at those indices. `--offset` looks into the middle of a run you have not done; `--isolate` draws a *different* sample when you want variety rather than fidelity.

25.3.2 `generate`

```
cacophony generate PROJECT [options]
```

GENERATE OPTIONS

OPTION	DEFAULT	EFFECT
`-n, --records N`	—	Override every entity's count. Repeatable, and `-n ticket=100000` overrides one entity
`--seed N`	project	Override the project seed
`-o, --output FORMAT`	`jsonl`	csv, json, jsonl, ndjson, parquet, sql, sqlite
`-d, --out-dir DIR`	`out`	Destination directory
`--output-profile NAME`	—	Write a layout declared under `outputs:` (chapter 10)
`-e, --entity NAME`	all	Generate one entity
`--batch-size N`	`1000`	Records per write batch
`--workers N`	`4`	Entities generated concurrently
`--provenance MODE`	`none`	none, run, record, field, full
`--on-failure POLICY`	`abort`	When a generator fails *or* a record fails validation: retry, skip, placeholder, incomplete, abort, report
`--drop-invalid`	off	Discard records that fail validation, whatever the policy says
`--no-validate`	off	Skip validation entirely
`--edge-cases FRACTION`	`0.0`	Fraction of records given a legal-but-awkward value
`--edge-categories LIST`	all	Limit edge cases to these categories
`--cache MODE`	`disabled`	disabled, read_only, read_write
`--cache-path FILE`	—	Where the generation cache lives
`-w, --world NAME`	—	Generate against a named world
`--assets-dir DIR`	`<out>/assets`	Where generated media goes
`--regenerate-assets`	off	Redraw media already on disk
`--llm-batch-size N`	`20`	Records per model call in batch mode
`--checkpoint-every N`	`10000`	Records between checkpoints
`--store FILE`	`.cacophony/`	Path to the run store
`--no-history`	off	Do not record this run
`--log-level LEVEL`	`warning`	debug, info, warning, error
`--log-format FORMAT`	`text`	text or json

25.3.3 regenerate

```
cacophony regenerate PROJECT -r N|N-M [-e ENTITY] [-c COLUMNS] [--json] [--seed N]
```

Re-derives specific records with no run and no dataset. Refuses more than a thousand and points at `generate`, which is the tool for that.

25.4 Runs

25.4.1 `runs`, `run`, `resume`

```
cacophony runs [-p PROJECT] [--store FILE] [--state STATE] [--limit N] [--
json] cacophony run RUN_ID [-p PROJECT] [--store FILE] [--events N] [--
json] cacophony resume [RUN_ID] [-p PROJECT] [--store FILE] [--log-level
LEVEL]
```

`run` accepts a unique prefix of an id. `resume` with no id takes the most recent resumable run.

25.5 Serving

25.5.1 `serve`

```
cacophony serve [--host H] [--port P] [--store FILE] [-p PROJECT] [--studio
DIR] [--allow-root DIR]... [--insecure] [--reload] [--log-level LEVEL]
```

The API, the live feed and the Studio, on `127.0.0.1:8765` by default. Interactive API documentation is at `/docs`. A bind that is not loopback requires `CACOPHONY_TOKEN` and confines every path a request names to `--allow-root`; chapter 37 explains why.

25.5.2 `desktop`

```
cacophony desktop [--store FILE] [-p PROJECT] [--studio DIR] [--host H] [--
port N] [--no-token] [--keep-running] [--log-level LEVEL]
```

The same application, with a handshake line on stdout and a lifetime tied to stdin. Chapter 39.

25.6 Providers and models

25.6.1 `providers`, `models`, `benchmark`

```
cacophony providers [PROJECT] [--test] cacophony models PROJECT [-p PRO-
VIDER] cacophony benchmark PROJECT -m MODEL,MODEL [-e ENTITY] [-n N] [-p
PROVIDER] [--sort-by FIELD] [--json] [--seed N]
```

25.7 Everything else

25.7.1 `begin`

```
cacophony begin "DESCRIPTION" [-o SCHEMA] [-d DIR] [-f FORMAT] [--scale N]
[--seed N] [-e] [-y] [--force] [provider options]
```

Propose a world, review it, build it. The chapter at the end of this part.

25.7.2 `propose`

```
cacophony propose "a description" [-o FILE] [--providers PROJECT] [--ad-
apter NAME] [--url URL] [-m MODEL] [--scale N] [--seed N] [--force]
```

The model proposes entities, fields and relationships; Cacophony picks the generators, compiles the result and lints it before showing it. What comes back is a schema known to work, and a file to edit.

25.7.3 `stream`, `cluster`, `controller`, `worker`

Chapters 29 and 30.

25.7.4 `transform`

Chapter 36.

CHAPTER

26

Providers

A provider is a backend that generates something Cacophony cannot compute: prose, pictures or speech. They are declared in the project, referenced by name from fields, and never hold a credential.

```yaml
providers:
  local_llm:
    type: language_model        # language_model, image, speech, custom
    adapter: ollama
    base_url: http://localhost:11434
    model: llama3.1:8b
    secret: my-secret-id        # a logical id, never a credential
    concurrency: 4
    timeout_seconds: 120
    options:
      structured_output: auto
```

26.1 Language-model adapters

LANGUAGE-MODEL ADAPTERS

ADAPTER	ALIASES	ENDPOINT	STRUCTURED OUTPUT
`ollama`	–	`/api/generate`	JSON Schema, enforced during decoding
`llamacpp`	`llama.cpp`, `llama_cpp`	`/completion`	JSON Schema, via a GBNF grammar
`openai_compatible`	`openai`, `vllm`, `lmstudio`, `tgi`	`/v1/chat/completions`	Negotiated: `json_schema`, then `json_object`, then prompt-only
`mock`	`fake`, `mock_llm`	none — in process	Synthesised from the schema

`openai_compatible` takes `structured_output`: `auto` (default), `json_schema`, `json_object` or `none`. `auto` discovers what the server supports on the first call and remembers the answer.

`mock` takes `failure_rate`, `malformed_rate`, `latency_ms`, `responses` and `healthy`. It is how you rehearse a run's shape without a GPU, and how you exercise the retry ladder deliberately.

26.2 Image and speech adapters

MEDIA ADAPTERS

ADAPTER	ALIASES	TYPE	NOTES
invokeai	invoke	image	Submits a graph and polls the queue. `queue_id`, `scheduler`, `steps`, `guidance`, `negative_prompt`, `poll_timeout_seconds`
procedural_image	procedural, placeholder_image	image	Draws in-process. `style`: identicon, portrait, card, document
piper	piper_http	speech	`POST /synthesize` — piper1-gpl's web server
openai_speech	openai_tts, speech_api	speech	`POST /v1/audio/speech` — openedai-speech, LocalAI, Kokoro-FastAPI
procedural_speech	tone, placeholder_speech	speech	Synthesises in-process. `sample_rate`, `voice`

YAML

```yaml
providers:
  pictures:
    type: image
    adapter: invokeai
    base_url: http://diffusion-box:9090
    model: Dreamshaper 8       # a name from the server's model list
    concurrency: 1             # one GPU, one request
    options:
      steps: 24
      guidance: 7.5
      poll_timeout_seconds: 300

  voices:
    type: speech
    adapter: piper
    base_url: http://tts-box:5000
```

The InvokeAI adapter builds a text-to-image graph for the named model's architecture; SD-1, SD-2 and SDXL are covered. FLUX, Qwen-Image and Z-Image each have their own node topology, so those need a `workflow` exported from InvokeAI — and the adapter says so by name rather than submitting a graph that will fail. A Piper server hosts one voice, chosen when it starts, so a schema wanting several voices points each at its own provider.

26.3 Checking them

```
TERMINAL
$ cacophony providers project.yaml --test
$ cacophony models project.yaml -p local_llm
```

Both adapters that talk to real media servers are verified against real servers by the live contract tests:

```
TERMINAL
$ CACOPHONY_TEST_INVOKEAI=http://diffusion-box:9090 \
  CACOPHONY_TEST_PIPER=http://tts-box:5000 \
  CACOPHONY_TEST_OLLAMA=http://gpu-box:11434 \
      pytest tests/test_provider_contracts.py -m live
```

26.4 The generation cache

`--cache read_write` stores model answers keyed by the prompt and the field's seed, so a re-run costs nothing and a nondeterministic generator becomes reproducible in practice. `read_only` uses what is there without adding to it. The benchmark forces the cache off, because with it on the second model would be scored on the first model's answers.

CHAPTER

27

Runs

Every `generate` records a run: what was asked for, what happened, how far it got. The store is SQLite, by default `.cacophony/cacophony.db` beside the schema file, and it never contains the generated data itself.

TERMINAL

```
$ cacophony generate project.yaml -o parquet -d out/   # ^C at any point
$ cacophony runs                                       # what has been run
$ cacophony run 4f2a91c3 --events 20                   # the inspector
$ cacophony resume 4f2a91c3                            # carry on
```

27.1 What the store holds

Projects, schema revisions, runs, jobs, checkpoints, events and statistics. Every accepted Studio edit creates a schema revision section 73, so a run is always associated with the exact text of the schema that produced it.

RUN AND JOB STATES

RUN	JOB
queued · running · paused · completed · failed · cancelled	queued · running · paused · retrying · completed · failed · cancelled

27.2 What a finished run reports

While a run executes, the numbers come from the process running it: records a second, tokens a second, bytes on disk, an estimate of what is left. When it stops, they come from what it stored — and they stop moving. A finished run's elapsed time is how long it took, not how long ago it was: the two are easy to confuse in an interface, and for a while this one confused them, reporting a minute and counting beside a duration of forty-two milliseconds.

Output size is measured from the files, not counted while writing them. A counter cannot know about a Parquet footer, a compressed page, a partition directory or a SQLite database that four entities shared, and the number

people want from "output size" is the size of the output. It is the sum of the run's own files, each counted once, taken after the writers close.

27.3 Checkpoints

A job checkpoints after every batch. A checkpoint is one integer — how many records that job finished — because a record's seed is derived from its index and there is no RNG state to serialise.

On resume the file on disk is *counted* and the checkpoint corrected to match it. A stale checkpoint would otherwise silently duplicate or skip records after an unclean stop, which is the classic way a resumable generator quietly corrupts a dataset.

27.4 Formats that cannot be appended to

A JSON array and a Parquet file cannot be extended in place, so a resumed run writes a new part: `employee.part0001.parquet`. Readers for both formats accept a directory of parts.

27.5 Resuming reuses the original configuration

The same seed, the same format, the same provenance mode, and the schema revision the run started with. Editing a schema and resuming would produce a dataset generated two different ways; start a new run instead. Cacophony will tell you if you try.

27.6 Events

The event log is structured and queryable, which is what makes a long run diagnosable after the fact:

KIND
`run.started` · `run.progress` · `run.paused` · `run.resumed` · `run.completed` · `run.failed` · `run.cancelled`
`job.started` · `job.progress` · `job.checkpoint` · `job.completed` · `job.failed`
`warning` · `error`

`--no-history` skips the store entirely, for throwaway runs and for environments where writing a database beside the schema is unwelcome.

CHAPTER

28

One sentence to a world

THE DESIGN DOCUMENT CLOSES WITH AN IMAGE RATHER THAN A FEATURE: SOMEBODY DESCRIBES the world they need in a sentence, Cacophony proposes it, they edit the proposal, and then they press a button marked BEGIN CACOPHONY. Every part of that existed separately. This is the flow that joins them.

```
TERMINAL
$ cacophony begin "a small hospital: staff, wards, and admissions over a year"
```

28.1 What it does

1. Asks a language model for a schema — the same assistant `propose` uses.
2. Writes it to a file.
3. Prints the world it would build: entities, record counts, model calls, media and an approximate size.
4. Asks.
5. Hands the compiled project to the same Conductor `generate` uses.

```
TERMINAL
──────────────────────────── CACOPHONY ────────────────────────────
building a small hospital: staff, wards, and admissions over a year

schema small-hospital.yaml
  note no primary key was proposed for: nurse

proposed 2 entities, 48 records
  ward                              8 records
  nurse                            40 records  after ward

  field values     136
  storage (approx) 1.8 KB

INFO  nurse
  No primary key is declared.

BEGIN CACOPHONY? [b]egin, [e]dit, [q]uit (b):
```

The linter runs on the proposal too, and its findings are part of what you are being asked about. Here it is telling you that `nurse` has no identifier — harmless for a leaf entity, worth a moment's thought if you were going to reference it later.

`e` opens the schema in `$EDITOR` and recompiles it when you come back, as many times as you like — a schema that does not compile puts you straight back in the editor rather than ending the session. `q` leaves the file where it is.

28.2 Two things it insists on

The schema is written before anything is generated. A world that only ever existed in one terminal session is an anecdote. What you get is a project file, a run recorded against it, and `cacophony generate <schema>` to do the whole thing again on another machine next year.

It asks. A sentence can quite easily describe four hours of work and a hundred gigabytes, so the estimate comes first and the question after it. `--yes` skips the question for a script, and outside a terminal it is skipped automatically — with a schema that lints clean required, since there is nobody there to fix one that does not.

BEGIN OPTIONS

OPTION	EFFECT
`-o, --out PATH`	Where the schema goes. Default: a filename made from the name the model gave it
`-d, --out-dir DIR`	Where the data goes
`-f, --format FORMAT`	Output format for the data
`--scale N`	Divide every proposed count by this
`--seed N`	Seed the world
`-e, --edit`	Open the editor before deciding anything
`-y, --yes`	Do not ask; begin as soon as it compiles
`--force`	Overwrite an existing schema file
`--providers FILE`	Borrow a project's language-model configuration
`--adapter, --url, -m`	Describe the model directly instead

> **THE BUG THIS FLOW FOUND ON ITS FIRST RUN**
>
> `primary_key` is not a required property of a proposal, and a model that omits it produces a schema whose references point at nothing: it compiles, it generates, and it fails on the first join. Cacophony now picks a key for any entity something references — `<entity>_id`, then `id`, then the first field ending `_id` — and marks it unique. "A schema that is known to work" has to include the joins.

28.3 What it is not

It is not a different generator, a different engine or a different set of guarantees. Every record it produces comes from the same pipeline as every other record in this manual, and the file it writes is an ordinary project you can edit, lint, bundle, stream, shard across eight machines, or throw away.

PART VIII

Scale

A workload rather than a file; a run spread across machines with byte-identical output; and what any of it costs.

CHAPTER

29

Streaming

THE SAME SCHEMA, DELIVERED CONTINUOUSLY AT A RATE YOU CHOOSE, UNTIL YOU STOP IT. This is Cacophony as a workload generator rather than a dataset generator, and the difference changes several behaviours in ways worth knowing.

TERMINAL

```
$ cacophony stream templates/security-operations.yaml \
      --rate authentication=250/s --rate security_finding=8/minute \
      --to syslog://siem.internal:514
```

29.1 Options

STREAM OPTIONS

OPTION	DEFAULT	EFFECT
`-r, --rate ENTITY=RATE`	—	Repeatable. `250/s`, `8 per minute`, `1200/hour`, or a bare number meaning per second
`-t, --to DESTINATION`	`stdout`	Repeatable; every destination gets every record
`-s, --seconds N`	forever	Stop after this long
`-n, --records N`	—	Stop after this many records
`--batch-size N`	`100`	Records per delivery
`--flush SECONDS`	`1.0`	Deliver a partial batch after this long
`--from N`	`0`	Start indices here, to continue a previous stream
`--follow-shape`	off	Modulate the rate by the timeline's shape: quiet at night
`--historical`	off	Keep generated timestamps instead of stamping with the wall clock
`--scenario-cycle SECONDS`	`3600`	How often a scenario window recurs
`--on-error POLICY`	`continue`	`continue` or `abort` when a destination rejects records
`--validate`	off	Check records against the schema and report failures; they are delivered either way
`-q, --quiet`	off	No dashboard; useful when piping

29.2 Destinations

`--to`

DESTINATION	FORM	NOTES
Standard output	`stdout`	One JSON object per line. The dashboard moves to stderr, so piping to `jq` works.
File	`file://path.jsonl`	Rotates by size; `rotate_bytes` and `keep` are options.
Syslog	`syslog://host:514` `syslog+tcp://host:601`	RFC 5424 by default, `rfc: 3164` available. TCP uses octet-counted framing.
HTTP	`https://host/ingest`	POSTs an ndjson batch per delivery; `array` and `single` body styles available.
Kafka	`kafka://broker:9092/topic`	Needs `pip install 'cacophony[kafka]'`. `key_field` partitions by a field.
Memory	`memory://200`	A bounded window of recent records for the API and the Studio to read back. Useless from the CLI, where stdout already does this.

29.3 What differs from a batch run

- **Indices keep going** rather than stopping at a count, so `--from N` continues a previous stream instead of replaying it. The summary prints the number to use.
- **Timestamps come from the wall clock**, because a stream is happening now. `--historical` keeps the generated ones.
- **Subjects interleave**, so per-subject state is held in memory rather than folded over a contiguous block.
- **Scenario windows recur** every `--scenario-cycle` seconds. An incident declared at `window: {at: 0.62}` has no meaning in an endless stream, so it happens again each cycle.

29.4 Nothing is validated unless you ask

A workload generator that stopped mid-load because one record in a million was invalid would have failed the test it was running, so `stream` does not check by default — and `--on-failure` has no effect here, deliberately. `--validate` turns the checks on: failures are counted and reported in the summary, and the records go to the destinations anyway.

TERMINAL

```
$ cacophony stream project.yaml -r event=200/s -s 30 --validate -t stdout -q
  rate            195.5/s of 200.0/s requested
  validation      77 of 80 failed
  stdout          80 delivered
```

29.5 Attainment

The dashboard shows the achieved rate against the requested one. Below 95% means generation or a destination could not keep up, and it is reported rather than hidden: a workload generator that quietly under-delivers is measuring the wrong thing.

> **ATTAINMENT IS INTEGRATED, NOT AVERAGED**
>
> When a rate is changed while the stream runs, the denominator is the integral of the requested rate over time rather than the rate at start-up. Without that, retargeting 250/s down to 100/s produced a reported attainment of 182% — a lifetime mean divided by the current target, which is a number about nothing.

CHAPTER

30

Distributed generation

A RUN CUT INTO SHARDS — CONTIGUOUS INDEX RANGES — AND HANDED OUT AS LEASES. The joined output is byte-identical to what one machine would have written, and that is a consequence of the seed design rather than an effort.

TERMINAL

```
# One machine, several workers
$ cacophony cluster templates/security-operations.yaml -o out/ --workers 8

# Several machines
$ cacophony controller templates/security-operations.yaml --port 8787
$ cacophony worker templates/security-operations.yaml \
      -c http://controller:8787 -o /mnt/shared
```

30.1 Everyone runs the same project

Every worker must run the same project file, and the same `--seed`, `--records` and `--format` as the controller. A worker whose schema hashes differently is refused at registration rather than allowed to contribute records from a different world.

30.2 Capabilities

Neither side declares anything. A shard's requirements are read off its compiled generators; a worker's are read off its configured providers.

CAPABILITIES

CAPABILITY	A SHARD NEEDS IT WHEN	A WORKER HAS IT WHEN
`deterministic`	always	always
`language_model`	a field uses `llm`	a `language_model` provider is configured
`image`	a field uses `image`	an `image` provider is configured
`speech`	a field uses `tts`	a `speech` provider is configured
`document`	a field uses `document`	always — rendering is in-process

`--capabilities deterministic,image` overrides detection, for a node whose model server is reachable but which you want kept off model work.

30.3 Leases

A shard is granted for `--lease-seconds` (default 30). The worker renews halfway through each interval while it generates. If a lease expires the shard is offered to somebody else, and the original holder is told its lease is stale and discards what it wrote. After `--max-attempts` failures (default 3) a shard is marked failed and the run reports it rather than retrying forever.

A re-leased shard is *regenerated*, not resumed. The second attempt produces exactly the bytes the first would have, so a dead worker's partial file is simply overwritten. This is why a shard needs no checkpoint: if it does not finish, it is simply done again.

30.4 Output

Each shard writes `entity.part<offset>.<ext>` — zero-padded, so an ordinary directory listing is already in dataset order. `jsonl`, `csv` and `json` parts join back into one file; `cluster` does it automatically and `--no-join` keeps the parts. `parquet` parts are left as a directory, which every reader for that format accepts.

> **SQLITE AND SQL ARE REFUSED**
>
> A relational output split across shards is not a relational output: its foreign keys would not resolve, because each shard knows only its own index range. Generate parts and load them, or use `cacophony generate`. Cacophony refuses with that explanation rather than producing databases that look fine and are not.

30.5 Shared artifacts

Point every worker's `--assets-dir` at one mounted directory. Assets are content-addressed, so two nodes producing the same file produce the same name with the same bytes. Each node appends to its own `manifest.<node>.jsonl`; readers read all of them as one run.

30.6 Authentication

Set `CACOPHONY_CONTROLLER_TOKEN` on the controller to require it, and the same variable on each worker to send it. It is read from the environment rather than taken as a flag, so it never lands in a shell history or a process listing.

30.7 Controller routes

METHOD	ROUTE	PURPOSE
POST	`/register`	A worker announces itself and its capabilities
POST	`/lease`	Ask for a shard
POST	`/renew`	Keep a lease alive
POST	`/complete`	Report a shard done
POST	`/fail`	Report a shard failed
GET	`/status`	Progress, per-worker health, throughput, reassignments, a `stalled` flag
GET	`/shards?state=…`	Shard states
GET	`/health`	Liveness

`/status` raises `stalled` when the remaining work needs a capability no live worker has — a cluster of deterministic-only nodes with model fields left to generate will otherwise sit there looking busy.

> **A COMPLETION THAT ARRIVES TWICE**
>
> The controller refuses a terminal lease. Without that, a `complete` resent after a network hiccup counted its records again and the run reported more output than it produced. Found by testing the protocol over HTTP rather than by reasoning about it.

30.8 What you give up

`generate` remains the single-node path, and it is the one with run records, checkpoints and resume. The distributed commands trade that bookkeeping for parallelism.

CHAPTER

31

Performance and memory

The design goal is that dataset size does not appear in the memory profile. It mostly does not, and the exceptions are listed here rather than discovered.

31.1 Everything streams

The engine yields bounded batches and writers flush them. A hundred million records cost what a hundred do, plus whatever the output format buffers. Measured on ordinary hardware: 124 MB and 400,000 records delivered in 5.4 seconds at 69 MB peak resident memory.

31.2 What does hold memory

FEATURE	HOLDS	BOUNDED BY
Near-duplicate detection	A sliding window of recent values	`quality.duplication.window`, default 50,000
Exact duplicate detection	A Bloom filter	About 18 MB for ten million values
Uniqueness validation	Seen values, then a disk-backed index	`--unique-memory`, default 250,000 values per field
Streaming simulation state	Per-subject state	Number of active subjects
Parquet writing	A row group	`--batch-size`
SQLite output	A transaction	`--batch-size`

31.3 The one that used to grow

Enforcing `unique: true` means remembering every value the field has produced, and a set of those is a structure whose size is the number of records — the one shape this chapter says nothing here has. A ten-million row run held ten million values for its whole duration.

It is bounded now. Values live in a set up to a stated ceiling; after that the set is written to a disk-backed index with a unique constraint and every later value is asked of that. Measured on a million short strings:

	PEAK MEMORY	TIME
Held entirely in memory	103.6 MB	3.9 s
Spilling at 250,000	25.9 MB	7.3 s

Four times less memory for twice the time, and the run says when it spilled so a slower run is explicable rather than mysterious. Most runs never reach the ceiling and pay neither.

> **EXACTNESS WAS NOT THE THING TRADED**
>
> A Bloom filter would be smaller and faster still, and would occasionally report a duplicate that is not one. A validator that cries wolf about a primary key is worse than no validator, so what is stored is the value rather than a digest of it: a collision cannot be mistaken for a repeat.

31.4 Tuning

KNOB	RAISE IT WHEN	LOWER IT WHEN
`--batch-size`	Writing Parquet or SQLite, where larger batches are much faster	Memory is tight, or you want finer checkpoints
`--workers`	Several independent entities and spare cores	One entity dominates; extra workers do nothing
`--llm-batch-size`	Prose volume matters more than nuance	Answers are drifting or being clipped
`unique_memory_values`	Memory is plentiful and a unique field is large	Memory is tight; the check moves to disk sooner
`--checkpoint-every`	Checkpointing shows up in the profile	An interrupted run must lose very little
`provider concurrency`	The model server has headroom	You have one GPU; set it to 1

31.5 What the estimate assumes

`cacophony plan` ends with an estimate, and section 69 is explicit that it must not pretend to be exact. Two of its figures depend on options rather than on the schema, so it takes them:

TERMINAL

```
$ cacophony plan project.yaml --batch-size 20000 --workers 8
```

Memory is a write batch, not a dataset: batch size times what a record costs in memory, times the number of entities in flight at once, which is `--workers` bounded by how many entities there are. The number moves with those flags because the run does. The Studio's generate screen does the same arithmetic against its own controls, so the figure on screen is about the run that button will start.

Model calls are counted the way they are made, not as records times model fields. In the default `per_record` mode, every model field in a layer that shares a provider travels in one call, so three of them cost one call per record and not three. `per_field` costs one each. `mode: batch` covers `--llm-batch-size` records per call — the estimate assumes the default of twenty and says so, in the plan's JSON as `assumed_llm_batch_size`.

31.6 Where the time actually goes

For a deterministic schema, in the writer and in the validator. For a schema with model fields, in the model — by two or three orders of magnitude. Before optimising anything, run `cacophony plan` and look at the cost classes: if any field says `llm` or `gpu`, nothing else in the pipeline is worth measuring.

Three ways to make model work cost less, in order of effect: use `mode: batch` with a large `--llm-batch-size`; turn on `--cache read_write` so a re-run is free; and reconsider whether the field needs a model at all — a weighted choice over forty written phrases is indistinguishable from a model's output in most datasets, and generates a million records a second.

PART IX

Assurance

Whether what came out is any good: is it valid, is it repetitive, does it have awkward corners, was the model up to it, and what to do when the answer is no.

CHAPTER

32

Validation

Generation and validation are separate stages on purpose. A generator that also decided what was acceptable could never be wrong; a validator that runs afterwards can be, and tells you.

32.1 What is checked

Section 57 names six categories, and all six are here. Four run always; two run when a schema asks.

CATEGORY	CHECKS
Structural	The value is of the declared type, or coercible to it without loss; nulls only where permitted
Constraint	`min`, `max`, lengths, `pattern`, `enum`, `forbidden`, `multiple_of`, `precision`, and `unique`
Referential	Every foreign key resolves to a record that exists
Statistical	Categorical fields land near their declared weights
Logical	An entity's `assertions:` — rules about a whole record
Semantic	A judge model's opinion of a sample. Off by default

32.2 Logical: rules about a record

A constraint asks whether a *value* is acceptable. Section 57's logical category asks whether a *record* makes sense, which takes more than one field to answer — the section's own example is `termination_date >= hire_date`.

```yaml
entities:
  employment:
    assertions:
      - expr: "ended_on == null or ended_on >= started_on"
        message: employment ended before it started
```

Evaluated with the same restricted evaluator as everything else in a schema, with `null`, `true` and `false` available as literals — the document around the expression is YAML, and a rule about a nullable field should not have to say

`None`. A failure is a validation failure of category `logical`, so `--on-failure` decides what happens next.

An assertion is part of the schema, so it is compiled when the schema is: `cacophony validate` refuses one that does not parse, uses a construct the evaluator does not allow, or names a field the entity does not have — and names the field it could not find. This used to wait for the first record, which reported a schema mistake as a data problem.

32.3 Semantic: an opinion, attributed

Off unless asked for. Section 57 says why — cost — and there is a second reason: judging generated text with a language model makes the measurement depend on the same kind of machinery that produced it, which is exactly the objection this manual records against scoring semantic quality in the benchmark (chapter 35).

```yaml
quality:
  semantic:
    enabled: true
    provider: local_llm
    sample: 25          # bounded, and stated
    threshold: 0.9
```

```
  semantic        note: 96% of 25 sampled values judged plausible by qwen3:8b
```

The model is always named. A plausibility rate with no attribution is worse than no rate at all, because it reads like a measurement.

An answer that is not a verdict is not counted as one. A judge that replies in prose, omits the field, or says something other than yes or no is recorded as `unreadable` and left out of the rate — because *n* unreadable answers scored as "implausible" is a broken judge reported as bad data. Words are read as words where they are unambiguous: `"false"`, `"no"` and `"No."` all mean no, which matters because a JSON string is true to any language that asks it a question carelessly.

32.4 Two different failures, one flag

A **generation** failure is a generator that could not produce a value at all — the model server is down, a template names a field that is null, a provider timed

out. A **validation** failure is a value that was produced and then found unacceptable. `--on-failure` governs both, and the default is to stop section 65.

--ON-FAILURE

POLICY	A GENERATOR THAT RAISES	A RECORD THAT FAILS VALIDATION
`abort`	Stop the run	Stop the run, naming the record and what was wrong with it
`retry`	Try the field again, up to the attempt limit	Generate the record again; if it still fails, drop it and count it
`skip`	Leave the field null	Drop the record and count it
`placeholder`	Write `[FAILED:field]`	Mark the offending fields `[FAILED:field]` and keep the record
`incomplete`	Write the record without the field	Remove the offending fields and keep the record
`report`	Leave the field null	Count it and write it anyway

`--drop-invalid` means `skip` for validation whatever the policy says, and `--no-validate` switches the checking off entirely — which is occasionally the right thing when you already know the data is deliberately broken and you want the bytes.

> **THIS USED TO BE TWO BEHAVIOURS**
>
> Until recently `--on-failure` governed only generation. A run that reported `validation failures 30,705` had written those thirty thousand invalid records to the file and returned success. It was found by running the tutorial in chapter 3 at a hundred thousand records, where a `unique` email drawn from a name list collides, and it is fixed: *abort* meaning "abort unless the data is merely invalid" is not a promise anybody can plan around.

32.5 What stopping looks like

TERMINAL

```
$ cacophony generate company.yaml -n 100000 -o jsonl -d out/

failed  0 records written
error EMP-000550 failed validation: email: duplicate value
'kenneth.cooper@example.com' for a field declared unique (uniqueness). Pass
--drop-invalid to discard records like this one, --on-failure report to write them
and count them, or --no-validate to stop checking.
  run             d7cc055f-d20d-40f9-97d9-8651c9748738

this is a schema problem, not an interrupted run - resuming would fail on the
same record.
$ echo $?
3
```

The run stops at the first invalid record. Whatever batches had already completed stay on disk and the run is recorded as `failed` with the count it reached — here the collision came at record 550 of a thousand-record batch, so nothing had been written yet.

It does not offer to resume, and that is the point of telling you which kind of failure this was: a record is a pure function of its index, so the second attempt would fail on the same one. The fix is the schema — that `unique` email cannot be drawn a hundred thousand times from a list of first and last names — or a policy that says carry on.

32.6 The three commands that report instead

`preview`, `regenerate` and `benchmark` all exist to *show* you a record that should not exist, so an invalid one is the answer rather than an error. All three still abort on a generator that raises, which is a different problem and still theirs to surface.

> **RETRYING ONLY HELPS WHERE A PROVIDER IS INVOLVED**
>
> A deterministic field reproduces the value that failed, so a retried record is the record that just failed. That is why an exhausted retry drops the record and counts it rather than stopping the run — which is also what an exhausted per-field retry does.

32.7 Damaged fields are skipped

Chaos marks what it damaged, and the validator does not check those fields. Otherwise every chaotic run would be a wall of failures reporting the feature working, and `--drop-invalid` would discard exactly the records you asked to be broken.

32.8 Quality scoring

A completed run carries referential and distribution scores, visible in the Studio and at `GET /api/runs/{id}/quality`: how many references resolved, how closely categorical fields matched their declared weights, and the duplication report if one was requested.

CHAPTER

33

Duplication

The characteristic failure of model-written data is repetition: the same three biographies, lightly reworded, forever. Measuring it needs care, because the naive approaches either miss it or cost more than the generation did.

YAML

```yaml
quality:
  duplication:
    max_exact: 0.001        # fraction of compared *values*, not records
    max_near: 0.02
    fields: [biography]     # default: the long-form text a model wrote
    methods: [exact, normalized, minhash]
    similarity: 0.7         # Jaccard at which two texts are the same thing
    shingle: 3              # word n-gram width
    window: 50000           # recent values held for near-duplicate comparison
    error_rate: 0.001       # Bloom filter false-positive target
```

Writing a threshold is a request to measure. Nothing is checked that nobody asked for, and `enabled: false` overrides that.

METHODS

METHOD	CATCHES
`exact`	Byte-identical values
`normalized`	The same text casefolded, depunctuated, whitespace-collapsed
`minhash`	Shared *phrasing* — a paragraph rewritten around one clause
`fuzzy`	The same candidates, confirmed with a real sequence ratio
`embeddings`	Named in the design and **refused**: it needs an embedding provider, and no adapter offers one

33.1 Which fields, by default

Model-written fields, fields declared `text`, and strings whose `max_length` is 80 or more. Comparing every employee id against every other one finds nothing

and costs a great deal; reporting that a weighted choice recurs would be reporting what a weighted choice is for. `["*"]` compares whole records.

33.2 What to believe

Exact matching uses a Bloom filter — 18 MB for ten million values rather than most of a gigabyte. It has no false negatives, so a report of zero is exact, and its false-positive rate at the load it actually reached is reported beside the count.

Near-duplicate detection holds a sliding window, because model repetition is *local*: the same three biographies come back within a few hundred calls, and a window catches that at any dataset size where a uniform sample would not.

33.3 The thresholds are calibrated, not guessed

WORD-TRIGRAM JACCARD ON A SIXTY-WORD BIOGRAPHY

RELATIONSHIP BETWEEN THE TWO TEXTS	SCORE
Identical but for the name	0.82
Name changed and one clause rewritten	0.69
Sharing an opening sentence and nothing else	0.13

Hence a default `similarity` of 0.7: above it the two texts are the same text, below it they merely start the same way.

> **A CONFIRMATION STEP THAT LIED**
>
> The fuzzy confirmation uses Python's `SequenceMatcher`, which by default treats characters appearing in more than one per cent of a long string as junk — that is, the vowels. A name-swapped biography scored **0.014** against a true value of 0.956. The autojunk heuristic is now switched off. It is recorded here because the failure was silent: the detector reported clean data, which is the worst possible way for a quality tool to be wrong.

33.4 What is excluded

Deliberate duplicates from `chaos`. Reporting those would be reporting the feature.

CHAPTER

34

Edge cases

Values the schema *permits* and naive code mishandles anyway. `O'Brien-Smith` is a real surname; an application that cannot store it has a bug, not bad input.

```
TERMINAL
$ cacophony generate project.yaml --edge-cases 0.05
$ cacophony generate project.yaml --edge-cases 0.2 --edge-categories
emoji,rtl_text
```

34.1 Not chaos

Entropy injection produces data the schema forbids and answers "what does my pipeline do with broken input". Edge cases produce data the schema permits and answer "is my code actually correct". They are separate flags because they are separate questions, and the answers have different owners.

34.2 The categories

`--edge-categories`

CATEGORY	VALUES
`boundary_length`	Empty string where legal, exactly `min_length`, exactly `max_length`
`punctuation_names`	`O'Brien-Smith`, `d'Arcy`, `van der Waals`, `Ó Séaghdha`
`unicode_text`	One-character names, ſ, Turkish dotted and dotless I, ligatures, fullwidth forms, Cherokee
`emoji`	Zero-width joiner sequences, family groups, flag pairs, skin-tone modifiers
`rtl_text`	Arabic, Hebrew, Persian, and the RLO override that renders reversed
`whitespace`	Leading, trailing, doubled, tab, non-breaking, zero-width, embedded newline
`extreme_numbers`	Declared bounds and one inside them; unbounded fields get 2^{31}, 2^{53}, 2^{63} and their negatives
`temporal_boundaries`	29 February, year end, the Unix epoch, and both US daylight-saving transitions

CATEGORY	VALUES
`extreme_coordinates`	The poles, the antimeridian, null island

34.3 Three rules that make the mode trustworthy

Every value is validated against the field that will hold it, and a candidate that does not fit is discarded and counted. An edge case that fails validation has told you nothing about your application.

Primary keys, unique fields and references are never touched. An emoji primary key is a broken fixture, not a robustness test: the joins stop resolving and every finding after that is about Cacophony.

Cases are applied as each field is produced, not to the finished record, so anything derived from an awkward value derives from the awkward value. A first name of `O'Brien-Smith` yields `o'brien-smith.smith@example.com`, which tests the template too. Doing it the other way round produced a colleague named `" leading space"` whose model-written biography still began "Courtney specializes in…" — two fields disagreeing is a broken fixture, not a finding.

Which records get a case, and which case, is derived from the record index, so a bug found once is found again.

> **BOUNDS ARE READ FROM THE GENERATOR, NOT JUST THE CONSTRAINTS**
>
> An early version proposed 2^{63} as an age, because the field had no `constraints.max` — only a generator option saying to draw between 21 and 67. The injector now reads the generator's own bounds as well, which is the difference between an edge case and nonsense.

CHAPTER

35

Benchmarking models

Not "which model is best" — a question with no answer — but "which of these models can do *this schema*", which has a measurable one.

TERMINAL

```
$ cacophony benchmark project.yaml -m gemma4:12b,qwen3:8b -n 100

MODEL               VALID  FIELDS  USABLE  CLIPPED   SPEED  DUPLICATION
LATENCY
gemma4:12b         100.0%  100.0%   91.7%        1  11 t/s         0.0%  1760
ms
smollm3:latest     100.0%  100.0%   70.0%        6  21 t/s         0.0%   806
ms
```

COLUMNS

COLUMN	MEANING
VALID	Answers that parsed without the repair stage touching them
FIELDS	Records that passed validation
USABLE	Values that were not empty, over-length, clipped or a refusal
CLIPPED	Values that stop dead at their length limit, mid-word
SPEED	Completion tokens per second
DUPLICATION	Repeated values, measured with the detector from the previous chapter
LATENCY	Mean per-call latency

35.1 Fairness is enforced, not documented as a caveat

Every model generates the same record indices from the same seed. The cache is forced off — with it on, the second model would be scored on the first model's answers and report an impossible speed. Concurrency comes from the provider spec and is stated, because 71 tokens per second at four concurrent requests is not comparable to 42 at one.

35.2 Why CLIPPED exists

A provider that enforces the JSON Schema natively stops decoding at `maxLength`, so the value is never *over* length — it is cut. `"…failed to handle queueing mechanism, led 3"` passes every check in the platform and is not a sentence. Without this column the benchmark could not fail: three models scored 100% on everything while producing output that was 70% usable.

35.3 What is not measured, and why

Semantic quality, which cannot be assessed without a judge model — which would make the benchmark depend on the thing it is assessing. What is measured instead is named honestly: empty values, broken lengths, clipped answers, and boilerplate about being a language model.

CHAPTER

36

Changing a dataset that exists

Sooner or later a dataset needs an edit: mask a column before it leaves the building, drop the test accounts, round the money. There are two ways to do it and only one of them survives regeneration.

36.1 The command

TERMINAL

```
# Rewrite a file. Streams, so the size does not matter.
$ cacophony transform out/employee.jsonl \
      --set 'email=mask:8' --where "department == 'Finance'" \
      -o out/employee.masked.jsonl --record-as mask_finance_emails

# Apply the project's own patch rules to a file.
$ cacophony transform out/employee.jsonl --project project.yaml --in-place

# Filter.
$ cacophony transform out/employee.jsonl \
      --keep-where "department == 'Engineering'" -o eng.jsonl

# Re-derive specific records, with no run and no file.
$ cacophony regenerate project.yaml -e employee -r 4823913-4823920
```

TRANSFORM OPTIONS

OPTION	EFFECT
`-s, --set FIELD=OP`	An operation pipeline for one field. Repeatable.
`-w, --where EXPR`	Only records matching this expression
`--drop-where EXPR`	Drop records matching
`--keep-where EXPR`	Keep only records matching
`-o, --out FILE`	Where to write the result
`--in-place`	Rewrite the file, via a temporary beside it
`-f, --format FORMAT`	`jsonl`, `csv` or `json`. Default: from the suffix
`-p, --project FILE`	Apply the project's own patch rules instead of the flags
`-e, --entity NAME`	Which entity's rules to apply
`--record-as NAME`	Also print this transform as a patch rule to paste into the project

Parquet cannot be transformed a record at a time: its records live in column chunks, so a row-by-row rewrite would mean decoding and re-encoding every one. Convert to JSON Lines, transform, convert back.

36.2 It never writes over its input

Including `--in-place`, which writes beside and swaps at the end. A rule that raises halfway through leaves the original untouched and no partial file behind. What it did is recorded in a `<name>.transform.json` sidecar, because a masked column is indistinguishable from a column that was always masked.

36.3 Patch rules, which are the better answer

```yaml
patches:
  mask_finance_emails:
    description: Finance addresses are masked before this dataset leaves the building.
    entity: employee                        # omit to apply to every entity
    where: "department == 'Finance'"        # omit to apply to every record
    set:
      email: "mask:8"                       # an operation pipeline
      notes: "upper(department)"            # or an expression over the record
      internal_ref: {value: null}           # or a literal
  drop_test_accounts:
    where: "startswith(email, 'test@')"
    drop: true                              # or `keep: true` for the inverse
```

A Cacophony dataset is a pure function of its schema and its seed. Editing a row in an output file breaks that silently: the file stops corresponding to anything, no other machine reproduces it, and the next `generate` overwrites the edit without noticing. A rule in the project applies on every run, travels with the schema, and `cacophony regenerate` produces the edited value a year later.

Rules apply in authored order, after chaos and before validation — so what the validator checks is what reaches the file, and a rule is the last word on what a record contains. Order matters: masking then hashing is a different value from hashing then masking.

`--record-as NAME` prints the equivalent rule so you can paste it in. Without it, `transform` warns that it changed a file and not the project, and that the next `generate` will produce the untransformed data again. That warning is the honest one.

36.4 Regeneration is nearly free

A record's seed is a hash of its position, so record 4,823,913 can be produced on its own — without the 4,823,912 before it, without the dataset, and without the run that made it. "This row looks wrong" is a question with a millisecond answer.

The Studio's data preview does the same thing from the other end: double-click a row and it builds a patch rule from it, showing the YAML rather than offering to save the row.

PART X

Interfaces

Three front doors onto one application: an HTTP API, a browser interface, and a window.

CHAPTER

37

The API

Everything the command line does, over HTTP, plus the live progress feeds the Studio is built on. One application serves the API and the interface, which is what keeps them from drifting apart.

```
TERMINAL
$ pip install 'cacophony[api]'
$ cacophony serve --port 8765
# interactive documentation at http://127.0.0.1:8765/docs
```

37.1 What serving exposes

On loopback this API is as powerful as the shell that started it. It registers projects *by path*, rewrites their schemas on disk, and writes runs to a directory the caller names. For a local tool that is the honest description rather than a flaw — the same shell could do all three — and confining it there would be theatre.

Bound to an interface other machines can reach, "the shell that started it" is somebody else's shell. So it refuses:

```
TERMINAL
$ cacophony serve --host 0.0.0.0
error refusing to serve on 0.0.0.0 without a token.
This API registers projects by path, rewrites their schemas, and writes
runs wherever a request asks. Off loopback that is somebody else's shell.

  export CACOPHONY_TOKEN=$(python -c 'import secrets;print(secrets.token_urlsafe(32))')
  cacophony serve --host 0.0.0.0 --allow-root .

Or pass --insecure if this interface is genuinely private.
```

SERVING BEYOND LOOPBACK

CONTROL	EFFECT
CACOPHONY_TOKEN	Required. Guards `/api` over HTTP and WebSockets — the same gate the desktop shell uses. From the environment rather than a flag, because a flag lands in shell history and in every process listing section 63
`--allow-root` PATH	Repeatable. Every path a request names must resolve inside one of these. Defaults to the project's directory, or the working directory

CONTROL	EFFECT
`--insecure`	Serve without a token anyway, for an interface you know is private

A refused path returns **403**, naming the directories that were permitted. Paths are resolved before they are checked, so `..` and a symlink both land where they really point rather than where they claim to.

Every path a request can name goes through that check, which is a list worth writing down: a run's output directory and assets directory, its provider cache file, a stream's `file://` destination, and a project's path — whether the project is registered from that file or supplies its schema inline and merely mentions it. Reads are checked as well as writes, so a project recorded before the server was confined cannot be read through it either.

> **THREE OF THOSE WERE ADDED AFTER AN AUDIT**
>
> The stream destination, the cache path, and the project path that arrives beside inline source. Each was the same mistake: a path reaches a component that opens it by a route where nothing checked it first. A confined server could be asked to append generated records to any file it could write, and to read back any file that parses as a project. They are checked now, and each has a test that performs the original request and expects a 403.

> **AND THE SCHEMA FILE IS REPLACED ATOMICALLY**
>
> Written beside the target, flushed, and renamed over it. A reader sees the old file or the new one and never a half-written one, and a crash costs the write rather than the project. `cacophony transform --in-place` always did this; the schema editor did not, which is the sort of inconsistency that only shows up on the day it matters.

37.2 Projects and schemas

METHOD	ROUTE	PURPOSE
GET	`/api/projects`	List registered projects
POST	`/api/projects`	Register one, by `path` or inline `source`
GET	`/api/projects/{id}`	A project with its schema revisions
GET	`/api/projects/{id}/plan`	The compiled generation plan
GET	`/api/projects/{id}/lint`	Linter findings
POST	`/api/projects/{id}/preview`	Sample records, with column sources
GET	`/api/projects/{id}/schema`	Source text plus the compiled shape
PATCH	`/api/projects/{id}/schema`	Targeted edits, preserving the document
PUT	`/api/projects/{id}/schema`	Replace the whole document
GET	`/api/schema/types`	Types and generators, for the editor controls

METHOD	ROUTE	PURPOSE
GET	/api/schema/operations	The edit operations and their arguments

37.3 Runs

METHOD	ROUTE	PURPOSE
POST	/api/projects/{id}/runs	Start a run
GET	/api/runs	List runs, filterable by state
GET	/api/runs/{id}	A run, plus live metrics while it executes
GET	/api/runs/{id}/jobs	Jobs and their checkpoints
GET	/api/runs/{id}/events	Structured event log
GET	/api/runs/{id}/quality	Referential, distribution and duplication scores
GET	/api/runs/{id}/assets	Generated files, filterable by kind and entity
GET	/api/runs/{id}/assets/file	One file, refusing paths outside the run
POST	/api/runs/{id}/pause	Pause at the next batch boundary
POST	/api/runs/{id}/resume	Unpause, or restart from checkpoints
POST	/api/runs/{id}/cancel	Cancel, checkpointing on the way out
DELETE	/api/runs/{id}	Delete a finished run
WS	/api/runs/{id}/stream	Live progress

37.4 Streams

METHOD	ROUTE	PURPOSE
POST	/api/projects/{id}/streams	Start a live stream
GET	/api/streams	Streams this server is running
GET	/api/streams/{id}	Rates, attainment, destinations, per-entity counters
GET	/api/streams/{id}/records	The bounded window of what it just produced
POST	/api/streams/{id}/retarget	Change one entity's rate while it runs
POST	/api/streams/{id}/pause	Hold, keeping the indices
POST	/api/streams/{id}/resume	Carry on
POST	/api/streams/{id}/stop	Stop, waiting for destinations to close
DELETE	/api/streams/{id}	Stop and forget
WS	/api/streams/{id}/feed	Status, pushed twice a second

37.5 Providers, plugins and the system

METHOD	ROUTE	PURPOSE
GET	/api/providers	Adapters, and a project's configured providers

METHOD	ROUTE	PURPOSE
GET	/api/providers/{id}/models	Models a provider serves
POST	/api/providers/{id}/test	Health check
GET	/api/generators	The generator registry
GET	/api/outputs	Registered formats, and a project's declared layouts
GET	/api/plugins	Installed plugins and what they contribute
GET	/api/system	Version and store statistics
POST	/api/system/prune	Remove finished runs older than a cutoff

37.6 Schema editing

A `PATCH` body is a list of operations applied as one transaction:

```json
{
  "operations": [
    {"op": "set_entity", "entity": "employee", "key": "count", "value": 10000},
    {"op": "set_field", "entity": "employee", "field": "biography",
     "key": "semantic", "value": "A short professional biography."}
  ]
}
```

EDIT OPERATIONS

OPERATION	ARGUMENTS
set_project	key, value
set_entity	entity, key, value
add_entity / remove_entity	name
set_field / unset_field	entity, field, key, value
add_field	entity, name, [value], [index]
remove_field	entity, name
rename_field	entity, field, name
move_field	entity, name, index
add_provider	name, [value]
set_provider	name, key, [value]
remove_provider	name

A `value` of `null` removes the key rather than writing `null`, because that is what clearing a control in a form means.

Edits are applied to the YAML document *in place*, so comments, ordering and formatting survive. The result must load and compile or the whole patch is re-

fused and the file is untouched. Every accepted patch creates a schema revision.

CHAPTER

38

The Studio

A browser interface over the same API: schemas, runs, live streams, assets, providers and plugins. Its distinguishing property is what it does *not* do — it never rewrites your schema file.

38.1 The pages

PAGE	WHAT IT IS FOR
Projects	Register, open and compare projects; schema revisions.
Studio	The schema editor: entity list, field editor, a graph of relationships, live preview and lint.
Generate	Start a run with the options `generate` takes — format, layout, entities, policies — and watch it.
Runs / Run	History, progress, events, jobs, quality scores, pause and cancel.
Stream	Start and steer a live stream; rates, attainment, destinations, recent records.
Assets	Generated media, filterable, with the manifest metadata beside each file.
Providers	Add, edit and remove providers; list their models; test that each one answers.
Plugins	What is installed, what it contributes, and anything that failed to load.
Settings	Store location, pruning, and the server's own version information.

38.2 In a smaller window

Cacophony is a desktop application and a browser tab beside a terminal, not a phone application, and the layout is built for that rather than redesigned for touch. Below about 860 pixels the navigation stops being a column and becomes a strip along the top, which is the difference between a third of a narrow window being chrome and none of it being chrome. Below about 520 the strip keeps the glyphs and drops the words, each link still carrying its name as a tooltip and as its accessible name. The Studio's three panes stack below 1,180.

Wide content scrolls inside its own panel rather than taking the page with it: a field table with eight columns gets a horizontal scrollbar, and the window does not. This is checked rather than asserted — the layout tests in chapter 12

load every page at four widths in a real browser and fail if the document ever extends past the viewport, or if a control ends up somewhere nobody can click.

38.3 Why it never rewrites your file

A form that saved by re-serialising its own model would produce a correct file with every comment and every deliberate ordering stripped out. Edits are sent as the targeted operations above and applied to the YAML document in place, so the only thing that changes is the thing you changed sections 48, 74.

38.4 Editing a field

The field panel is section 49's, in its order: name, type, meaning, generation, context, length, tone, null probability, and a button that generates samples. What it shows follows the field — a length constraint on a boolean and a temperature on a sequence are noise, so neither appears.

Structured options are edited as what they are rather than as text. A weighted choice is a list of value-and-weight rows, each showing the share that weight is actually worth, so you are not doing the arithmetic in your head while editing five of them; a list of values is one per line; anything with more shape than that is JSON, parsed when you leave the box and reported beside it if it will not parse, rather than sent to be refused section 49.

38.5 Field order is an edit

Field order is the order of the columns in the output, so moving a field is a change to the schema, not a view preference. Drag a row in the field list to move it; or focus a row and press `Alt` with the up and down arrows, since a pointer gesture is not an interface on its own. Both send the same `move_field` operation the API takes section 48.

38.6 Live preview

The field editor previews as you type, using the same positional-seed mechanism as `cacophony preview`. Because previewing disturbs nothing, this is safe to do continuously against a project that is mid-run.

38.7 The generate screen

Section 54's list, in order: requested scale, estimated workload, provider requirements, disk estimate, the plan and the warnings — then one button. Three parts of it are worth knowing about.

The format menu is the writer registry. Every registered format is offered, database formats included, with the extension and the one-line description each writer carries. A format registered by a plugin appears without anything in the interface knowing it exists. Choosing SQLite says so: every entity becomes a table in one `cacophony.db`, rather than a file each.

A layout is chosen by name. If the project declares `outputs:` (chapter 3), the layouts appear in a menu above the directory. Choosing one fills in the format, the directory and the entity selection it decides, and shows what it will do with them — including the partitioning, which has no control of its own. The controls stay editable afterwards, so an edit is an override rather than a contradiction: the same precedence `--output-profile analytics -d /tmp/scratch` has on the command line.

Provider requirements are requirements, not an inventory. The screen reads which fields need a language model, an image model or speech, and names the configured provider serving each. Anything needed and unserved is called out before the run rather than after it — a field with `on_unavailable: placeholder` does not fail, it quietly emits marked stand-ins, which is the failure worth catching early.

The estimate uses section 69's own units: completion tokens rather than a guess of so-many-per-call, and the memory the run holds at once, which is a write batch and not the dataset sections 31, 69. Both scale with the record override as you type it, because an estimate for the schema's declared counts while the form says something else is not inexact, it is wrong.

38.8 Configuring a provider

A provider is four lines of YAML that people get wrong once and then avoid, so the Providers page edits them: add one by name and adapter, set its URL, model, concurrency and timeout, or remove it. Each change is one targeted patch, so the comments around it survive.

Three things are refused rather than written. An adapter this build does not have — because the misspelling compiles perfectly well and then decides, at run time, that every language-model field is unavailable. A provider still named by a field, which the message lists. And anything in the secret field that looks like a credential: a project file holds a logical secret id, resolved from an

explicit override, the environment or the OS keychain when the run starts, and never the credential itself section 63.

The adapter menu also keeps the declaration honest: changing the adapter changes the declared `type:` with it, in the same patch. Nothing dispatches on that field — but it is the one place a person would look to find out what a provider is for, and a provider labelled a language model while serving images is a lie in exactly that place.

CHAPTER

39

The desktop application

A Tauri window hosting the Studio, with the backend as a child process. There is no desktop mode in the backend: `cacophony desktop` serves the same application `cacophony serve` serves.

```
TERMINAL
$ ./desktop/build.sh            # a shell that uses `cacophony` from PATH
$ ./desktop/build.sh --bundle   # an installer, with the backend frozen inside
```

39.1 Which command opens a window

Not `cacophony desktop`. That is the backend half: it prints a handshake line and then serves until its stdin closes, which is what a shell wants and looks like a hang to a person. The window is the Tauri binary, and it spawns `cacophony desktop` itself.

```
TERMINAL
$ ./desktop/src-tauri/target/release/cacophony-desktop   # the window
$ cacophony desktop                                      # the backend it
spawns
$ cacophony serve
# the same thing, in a browser
```

Run by hand in a terminal the backend says so, and offers an address to open in a browser instead — token included, since `/api` refuses without it:

```
TERMINAL
$ cacophony desktop
CACOPHONY_HANDSHAKE {"version": 1, "url": "http://127.0.0.1:32781", "token": "ki2NQ…", "pid": 575273}

This is the backend for the desktop window, not the window itself.
To open one:
    /home/jason/Cacophony/desktop/src-tauri/target/release/cacophony-desktop
or open this in a browser - the token is what gets you in:
    http://127.0.0.1:32781/?token=ki2NQkmGRPfWuozyMTGgmMHnjDpRVAv0Y_2jUCgs-N4
Ctrl-C to stop.
```

Only when a person is watching. The shell spawns it with pipes rather than a terminal, so it sees nothing but the handshake — which is the line stdout is reserved for either way.

39.2 The handshake

TERMINAL

```
$ cacophony desktop
CACOPHONY_HANDSHAKE {"version":1,"url":"http://127.0.0.1:41287","token":"…","pid":8123}
```

One line of JSON on stdout, then the server runs. The shell spawns that, reads the line, and opens a window at the URL. Any other line the backend prints is its own logging and is passed through.

FOUR PROPERTIES, AND WHY EACH ONE

PROPERTY	WHY
A port the OS chose	A fixed 8765 collides with the `cacophony serve` somebody already has running.
Printed after binding	A shell that opened a window on a guessed URL would work on fast machines and fail on slow ones — the worst kind of bug.
A per-launch token	A loopback server is reachable by every process on the machine. A browser tab is an explicit act by a person; a window is not.
Dies when stdin closes	The one shutdown path that survives the shell being `SIGKILL`ed, which no signal handler does.

39.3 The token

It guards `/api` over **both** HTTP and WebSockets. The Studio itself is served unauthenticated: static files carrying no data, and the window has to load before it can present anything. The token travels in the query string once; the page reads it, stores it, and strips it from the address bar so it does not end up in a screenshot.

> **THE HOLE THAT A WEBSOCKET WALKS THROUGH**
>
> The first version of the gate used Starlette's HTTP middleware, which only sees `scope["type"] == "http"`. A WebSocket handshake passed straight through it, leaving the two socket routes open while every other route was guarded. A gap in a security control is worse than no control, because the control is what stops anyone looking. The gate is pure ASGI for this reason, and it was found by connecting a socket rather than by reasoning about the code.

39.4 Building a release

The shell is the easy half; the runtime is the hard one. A user who double-clicks an icon must not need Python, so a bundle freezes an interpreter and the backend into one executable with PyInstaller, which the shell finds beside itself as `cacophony-backend`. `CACOPHONY_BACKEND` overrides that, which is how a checkout points at its own virtualenv.

Neither half cross-compiles — Tauri needs the platform's own webview and PyInstaller needs the platform's own interpreter — so a release is built once per platform. The repository's workflow does that for Linux, macOS and Windows, and smoke-tests each frozen backend by reading its handshake.

> **SIGNING IS NOT CONFIGURED, AND IS DELIBERATELY NOT FAKED**
>
> macOS notarisation needs an Apple Developer identity and Windows needs a code-signing certificate; both are secrets a repository owner supplies. A workflow that pretended to sign would produce installers that fail on first launch with a message about an unidentified developer.

39.5 Web deployment is unaffected

`cacophony serve` passes no token and behaves exactly as it always has. The design document requires that web deployment remain possible, and the cheapest way to guarantee it is to have no second application to keep in step.

PART XI

Extending

How other people's code gets in — and the one door that is deliberately nailed shut.

CHAPTER

40

Plugins

A plugin is an installed Python package that registers into extension points Cacophony already had. Eight categories, one manifest, and a discovery rule that is the whole security model.

```
TERMINAL
$ cacophony plugins
$ cacophony plugins --show network_packets
$ cacophony plugins --json
```

40.1 Discovery is by installed entry point, never from a project directory

This is the decision the feature turns on. A schema arrives by email, in a Git repository, inside a `.cacophony` bundle. If Cacophony loaded Python from a `plugins/` folder beside a schema, opening one somebody sent you would be equivalent to running their code — and every other safety property in this manual would be decoration: the expression allow-list, the bundle importer's refusal of traversal, all of it.

The trust decision belongs to a person running `pip install`, not to a program opening a file.

40.2 Writing one

```toml
# pyproject.toml
[project.entry-points."cacophony.plugins"]
network_packets = "my_package:NetworkPackets"
```

PYTHON

```python
from cacophony.core.interfaces import SyncGenerator
from cacophony.generation.generators.base import OptionsMixin


class NetworkPacketGenerator(OptionsMixin, SyncGenerator):
    deterministic = True

    def prepare(self):
        self.protocol = self.opt_choice("protocol", ("tcp", "udp"), "tcp")

    def generate_sync(self, context):
        rng = context.rng()
        return f"{self.protocol}/{rng.randrange(1024, 65536)}"


class NetworkPackets:
    manifest = {
        "name": "network_packets",
        "version": "1.0",
        "provides": {"generators": ["network_packet"]},
    }

    def register(self, host):
        host.add_generator("network_packet", NetworkPacketGenerator, aliases=("packet",))
```

40.3 The eight categories

Each reaches a registry that already existed — the generator registry since the first phase, providers since the second, output formats since the first, transform operations since the twelfth. A plugin is a door into an extension point, not a mechanism beside one.

PLUGIN CATEGORIES

CATEGORY	HOST METHOD	REACHES
generators	add_generator(name, cls, aliases=())	The generator registry
transforms	add_transform(name, fn)	The operation set
outputs	add_output(name, writer)	The output formats
validators	add_validator(name, cls)	Validation categories
scenarios	add_scenario(name, cls)	Scenario behaviours
language_models	add_language_model(name, cls)	The provider registry
images	add_image_provider(name, cls)	The provider registry
speech	add_speech_provider(name, cls)	The provider registry

40.4 The manifest is a contract, checked both ways

Registering something the manifest did not declare has it *refused*; declaring something and never registering it is reported as *missing*. Neither is a security measure — a plugin is code you installed — but a manifest that has drifted from its code produces a project that works on one machine and fails on another with no clue why.

A plugin may not take over a name that already exists unless it sets `replace = True`, because one that silently replaced the built-in `uuid` generator would change every project on the machine.

40.5 A broken plugin does not stop a run

One that raises on import is recorded with its error and skipped: a plugin installed last month should not stop today's generation of a project that does not use it. A project that *requires* it does stop, at compile time, by name:

```yaml
requires:
  plugins: [network_packets]
```

`CACOPHONY_NO_PLUGINS=1` skips loading entirely — for a run that must be reproducible against the built-ins alone, or for bisecting a plugin problem.

CHAPTER

41

The `script` generator

The design document lists a `script` generator: "a user-provided generator run in an isolated environment". It is declared, it compiles, and it is deliberately not implemented. This chapter is the reasoning, recorded because a future reader will want to reopen the question.

41.1 The requirement is absolute

A project file is shared. If a `script:` field ran, opening a schema somebody sent you would be equivalent to running their code. That is not a risk to be mitigated with care; it is the thing that must not happen.

41.2 A restricted interpreter is not a sandbox

Stripping `__builtins__` and denying imports is a denylist, and denylists on a language with introspection are routinely escaped through object graphs nobody thought about. The technique has a decades-long record of being broken by people who were not even trying hard.

41.3 What isolation is available was measured, not assumed

Unprivileged user and network namespaces do work on Linux — a subprocess in one cannot open a socket, which was verified rather than supposed — but the filesystem stays readable, and blocking it needs a mount namespace and a pivot into an empty root. That is Linux-only machinery, untestable on the macOS and Windows the desktop phase targets.

A security boundary that exists on one of three platforms is not a boundary. It is a false sense of one, which is worse than none, because it invites the behaviour it cannot protect.

41.4 WebAssembly is the honest option and is out of scope

A CPython build for a WASM runtime gives real isolation, with ceilings enforced by the runtime rather than by a list. It is also a multi-megabyte dependency and substantial work, and it is how this should be done when it is done.

41.5 Use instead

IN ASCENDING ORDER OF POWER

INSTEAD OF A SCRIPT	USE	BECAUSE
A derived value	`expression`	An allow-listed evaluator with the functions you actually wanted
A per-record transformation	`patches`	Rules that travel with the schema and survive regeneration
Arbitrary code	A plugin	A package you chose to install, which is where that decision belongs

A `script` field still compiles, lints, plans and estimates, and `on_unavailable: placeholder` runs the whole pipeline with a marked stand-in — so a schema written against a future in which this exists is not a schema that fails today.

PART XII

Operating

What goes wrong, what it means, and the habits that keep a synthetic dataset trustworthy.

CHAPTER

42

Troubleshooting

Symptoms, in the order people meet them.

42.1 The schema will not compile

MESSAGE	USUALLY MEANS
Unknown generator `x`	A typo, or a plugin that is not installed. `cacophony generators` lists what exists.
Option `x` is required	A generator needs something you did not give it — `reference` without `entity`, `tts` without `source`.
Cycle: `a → b → a`	Two fields each depend on the other, usually through templates. Break it with a literal or a different source field.
Provider `x` is referenced but not configured	A field names a provider with no entry under `providers:`.
Plugin `x` is required and not installed	`requires.plugins` names something absent. This is deliberate: better now than three million records in.

42.2 The run stops on a validation failure

The default policy is `abort`, so the first invalid record ends the run and names itself, the field and the value. Three questions, in order: is the constraint right; is the generator producing what the constraint expects; and is a model involved, in which case the answer is usually a length limit that is too tight. See the note on clipping in chapter 15.

Resuming will fail at the same record, because the record is a pure function of its index. Fix the schema, or say what should happen instead: `--drop-invalid` to discard those records, `--on-failure report` to write them and count them, `--no-validate` to stop checking. Chapter 32 has the whole table.

A common one in a schema that grew: a `unique` field whose domain is smaller than the record count. Two hundred and fifty employees drawn from Faker's name lists produce distinct addresses; a hundred thousand do not, and

the run reports thirty thousand duplicates. `cacophony lint` catches the arithmetic cases before the run; the collision cases only show up at scale.

If the data is *meant* to be broken, you want `chaos:` and not a validation failure; chaos-damaged fields are skipped by the validator precisely so that this distinction stays clear.

42.3 A database output fails to load

If `chaos` is enabled, the tables are written without constraints by design, and a loader expecting them will complain. Generate without chaos for a database whose constraints hold. If chaos is not enabled and a foreign key still fails, check for a self-reference that points forwards — a Cacophony-generated self-reference always points backwards, so this indicates a hand-written key column rather than a `reference` generator.

42.4 The dataset does not look real

Four culprits, in the order they matter:

1. **Uniform references.** Set `distribution: skewed` and a `skew` around 1.9. Nothing else changes a dataset's character as much.
2. **Uniform timestamps.** Use a `timeline:` with a `shape`, so nights and weekends are quiet.
3. **Uniform categories.** A `weighted` generator with real-looking weights instead of `random`.
4. **Repetitive prose.** Turn on duplication measurement and look at the number; if it is high, the model is looping and needs more context or a higher temperature.

42.5 Generation is slow

Run `cacophony plan` and read the cost classes. If any field is `llm` or `gpu`, that is where all the time is going and nothing else is worth measuring. See chapter 31.

42.6 A resumed run refuses

The schema has changed since the run started. That is not obstinacy: a dataset generated half under one schema and half under another is not one dataset. Start a new run.

42.7 The desktop window says the backend is missing

The shell looks for `CACOPHONY_BACKEND`, then a bundled `cacophony-backend` beside the executable, then `cacophony` on `PATH`. In a checkout, point it at the virtualenv:

```
TERMINAL
$ CACOPHONY_BACKEND=$PWD/.venv/bin/cacophony \
      ./desktop/src-tauri/target/debug/cacophony-desktop
```

42.8 The server says the Studio has not been built

A page at `/` explaining that, rather than the `{"detail":"Not Found"}` it used to answer, which said the same thing in a way nobody could act on. It means no Studio is mounted: it is built by `npm --prefix frontend run build` and written into the package, and a fresh clone has none, because generated output is not committed. Older builds returned a bare 404 here.

If you installed Cacophony from a wheel rather than from a clone, there is no `frontend/` to build: the wheels deliberately carry no Studio assets (chapter 1). Use the API, or work from a clone.

If you *have* built it and still get this, check which package is being imported:

```
TERMINAL
$ python -c "import cacophony; print(cacophony.__file__)"
```

It should print a path under `backend/`. If it prints one under `site-packages/`, the install is a *copy* rather than an editable one — and that copy has no Studio in it, and no code you have edited since. `pip install -e .` puts it back.

42.9 The window shows "WebKit encountered an internal error"

That is the webview's web process failing, and there are two quite different reasons for it.

The likelier one is that the shell never got a backend at all. It looks for `CACOPHONY_BACKEND`, then a bundled `cacophony-backend` beside itself, then `cacophony` on `PATH` — and a terminal without the virtualenv activated has none of the three. It then opens a window built from a `data:` URL to explain itself, and if *that* will not render, the only thing you see is WebKit's error page. The reason is printed to stderr as well now, so a terminal always tells you. Point it at the backend explicitly:

```
TERMINAL

$ CACOPHONY_BACKEND=$PWD/.venv/bin/cacophony \
      ./desktop/src-tauri/target/release/cacophony-desktop
```

The other two are environmental, and the shell now defends against both itself. They are recorded here because the symptom names none of them.

42.9.1 The DMA-BUF renderer segfaults

WebKitGTK's DMA-BUF path crashes the web process on a good number of Linux graphics stacks. Measured here on an AMD card with Mesa under Wayland, and on a virtual X display with no DRI3: both segfault with it on, both run with it off, and forcing software GL does not help — which is what places the fault in that renderer rather than in the driver under it. The shell sets `WEBKIT_DISABLE_DMABUF_RENDERER=1` at startup unless you have set the variable yourself, so a machine where the fast path works can still use it.

42.9.2 A snap's libraries, borrowed by accident

Launching the window from a terminal inside a snap-confined editor — VS Code's integrated terminal being the common one — inherits that snap's environment. `GTK_PATH`, `LOCPATH`, `GIO_MODULE_DIR` and friends all point into `/snap`, WebKit's helper processes load that snap's C library in preference to the system's, and the network process dies:

```
WebKitNetworkProcess: symbol lookup error:
  /snap/core20/current/lib/x86_64-linux-gnu/libpthread.so.0:
  undefined symbol: __libc_pthread_init, version GLIBC_PRIVATE
ERROR: WebKit encountered an internal error. This is a WebKit bug.
```

It is not a WebKit bug. It is a program being handed another application's C library. The shell drops any of those variables whose *value* points into a snap, unless its own executable lives there too — in which case the paths are its own and are correct.

TEST THE VALUE, NOT THE MARKER

The first version of this checked whether `SNAP` was set, and did nothing unless it was. That is wrong in precisely the case that matters: VS Code's *integrated terminal* drops the `SNAP_*` bookkeeping while keeping `GTK_PATH` and `LOCPATH`, so the environment is poisoned without announcing itself. It passed its own testing because the shell it was tested from happened to have `SNAP` set — which is a good argument for reproducing the reporter's environment rather than one that resembles it.

IF YOU ARE ON AN OLDER BUILD

Both defences are in the shell binary, so a stale one still fails. Rebuild with `cargo build --release` in `desktop/src-tauri`, or launch from a terminal outside the snap and with `WEBKIT_DISABLE_DMABUF_RENDERER=1` set.

On Ubuntu 24.04 and later there is a third possibility worth ruling out: `kernel.apparmor_restrict_unprivileged_userns=1` stops WebKit's sandbox starting its web process. `sysctl` says whether it is on.

CHAPTER

43

Operating notes

Habits that keep a synthetic dataset defensible six months later, and a short list of the things worth putting in a runbook.

43.1 Reproducibility checklist

- Pin `project.seed` explicitly.
- Record the schema, not the output — a project is kilobytes and a dataset is gigabytes.
- Prefer a named world to a bare seed when several datasets must describe the same population.
- Keep edits as `patches:` rules, never as edits to output files.
- Use `cacophony bundle export` to hand the whole thing to somebody else, and `bundle inspect` on anything you receive.
- Run `cacophony lint --strict` in CI, and leave `--on-failure` at `abort` there: a pipeline that generates invalid data should fail rather than publish it.
- Note the Cacophony version alongside the dataset; a generator's behaviour is part of what reproduces it.

43.2 Environment variables

VARIABLE	EFFECT
`CACOPHONY_SECRET_<ID>`	Resolves a provider's logical secret
`CACOPHONY_CONTROLLER_TOKEN`	Shared token between a distributed controller and its workers
`CACOPHONY_NO_PLUGINS`	`1` skips plugin loading entirely
`CACOPHONY_BACKEND`	Which backend executable the desktop shell spawns
`CACOPHONY_TEST_OLLAMA`	Base URL for the live provider contract tests
`CACOPHONY_TEST_INVOKEAI`	As above, for image generation
`CACOPHONY_TEST_PIPER`	As above, for speech

43.3 Where things live on disk

PATH	WHAT
`.cacophony/cacophony.db`	The run store, beside the schema file. Never contains generated data.
`<out-dir>/<entity>.<ext>`	The records
`<out-dir>/assets/`	Generated media, with `manifest.jsonl`
`<out-dir>/<entity>.part<n>.<ext>`	Parts from a resumed or distributed run
`<file>.transform.json`	What a transform did to a file

43.4 Handing data to somebody else

Three things to check before a synthetic dataset leaves the building, none of which Cacophony can decide for you:

1. **Remove the ground truth.** A `scenario` field is the answer key. Leaving it in a dataset given to a detection team makes the exercise meaningless.
2. **Say that it is synthetic.** In the filename, in the manifest, in the ticket. Cacophony labels generated media `synthetic: true` and cannot label your CSV for you.
3. **Check the identifiers are still safe.** If any field carries `safe: false`, know why, and know that those values look real because they are shaped exactly like real ones.

43.5 Testing your own changes

TERMINAL

```
$ ruff check backend tests && ruff format --check backend tests
$ mypy backend/cacophony
$ pytest tests/
# <!-- claim:tests -->1771<!-- /claim --> tests, about 90 seconds
$ npm --prefix frontend run typecheck && npm --prefix frontend test
$ npm --prefix frontend run test:e2e  # the layout, in a real browser
```

The live provider contract tests are marked and skipped unless the `CACOPHONY_TEST_*` variables point at real servers, so the default suite needs no GPU and no network.

The layout tests are separate because they need a browser (`npx playwright install chromium`, once). They run the Studio at four window widths with the API intercepted, and check the two things a component test in jsdom cannot

see: that the page never scrolls sideways, and that every control stays somewhere a person can reach. jsdom has no layout at all — it will happily report that a 216-pixel sidebar and a 1,200-pixel table both fit inside a 700-pixel window.

PART XIII

Appendices

The tables. Every generator and its options, the whole recipe catalogue, the operations, the schema keys, the commands, the lint rules, and an index of everything with a name.

APPENDIX

A

Generator quick reference

All thirty, with their options. Chapters 11–16 explain what each is for; this is the page to keep open while writing a schema.

GENERATORS AND THEIR OPTIONS

GENERATOR	OPTIONS
constant	value
sequence	format · start · step · pad
uuid	version · namespace · name · string
random	min · max · precision · length · min_length · max_length · charset
boolean	probability
weighted	choices
distribution	distribution · mean · stddev · rate · scale · lam · alpha · beta · bins · min · max · precision
lookup	values · path · column · mode
pattern	pattern · upper · lower
template	template · on_missing
expression	expression
composite	steps
transform	operations · source
null	—
datetime	start · end · business_hours · weekdays_only · timezone_offset
ip	network · version · safe
mac	oui · separator · upper · safe
phone	format · area_code · safe
government_id	masked · safe
faker	provider · locale · unique · safe · *plus anything Faker takes*
reference	entity · field · distribution · skew · unique · expose · on_unavailable
event_time	jitter · offset · offset_field · spread
subject	field
state	variable
scenario	report · normal
llm	provider · mode · context · max_tokens · temperature · on_unavailable

GENERATOR	OPTIONS
image	prompt · style · width · height · steps · guidance · negative_prompt · workflow · provider · reuse · on_unavailable
tts	source · voice · voice_field · speed · language · sample_rate · provider · reuse · on_unavailable
document	template · template_path · format · title · page_size · font · font_size
script	on_unavailable *(declared, not implemented)*

APPENDIX

B

The recipe catalogue

Thirty-one fragments in five groups. `includes` lists the other recipes a recipe pulls in; its own fields are listed beside it. Every one is asserted to compile, generate and validate in the test suite.

BUILT-IN RECIPES

RECIPE	WHAT IT IS	FIELDS	INCLUDES
IDENTITY			
`address`	A postal address, in documentation-safe form.	`street, city, postcode, country`	—
`customer`	Someone who has bought something - name, contact, tier, signup.	`customer_id, tier, signed_up_at, marketing_opt_in`	`person, email, address`
`email`	A work address on a reserved domain, derived from the name.	`email`	`name`
`employee`	Section 80's "US Corporate Employee Identity": name, employee id, email, username, department, and the manager relationship. *Needs nothing - the manager reference points at this entity.*	`employee_id, department, seniority, hire_date, manager`	`person, email, username`
`name`	A given name and a family name.	`first_name, last_name`	—
`person`	A human being - name, date of birth, contact details.	`full_name, date_of_birth, phone`	`name`
`username`	A login name derived from a person's name.	`username`	`name`
COMPUTING			
`browser`	A browser and a plausible major version.	`browser, browser_version`	—
`device`	A managed endpoint - name, hardware, addresses, compliance.	`device_id, device_type, manufacturer, serial_number, last_seen_at, encrypted`	`hostname, ip, mac, os`
`hostname`	A machine name in a documentation domain.	`hostname`	—
`ip`	An IPv4 address from RFC 5737 documentation space.	`ip_address`	—

RECIPE	WHAT IT IS	FIELDS	INCLUDES
`mac`	A hardware address from RFC 7042's documentation OUI.	`mac_address`	—
`os`	An operating system and its version, weighted like a real fleet.	`operating_system`	—
`software_version`	A semantic version, weighted so most installs are current.	`version, release_channel`	—
SECURITY			
`alert_severity`	An alert severity and priority, weighted like a real queue.	`severity, priority`	—
`cve_identifier`	A CVE-shaped identifier in a year that has none published.	`cve_id`	—
`cvss_score`	A CVSS-like base score, distributed the way real ones are.	`cvss_score, cvss_severity`	—
`hash_value`	SHA-256-shaped and MD5-shaped digests, drawn rather than computed.	`sha256, md5`	—
`logon_event`	An authentication attempt - who, from where, and how it went. *Needs a timeline, and a `user` entity if the subject should be a reference.*	`event_id, timestamp, logon_type, source_ip, result, mfa_used`	—
`network_event`	A connection record - endpoints, protocol, bytes, verdict.	`flow_id, source_ip, destination_ip, destination_port, protocol, bytes_sent, action`	—
COMMERCE			
`currency`	An ISO-4217 currency code, weighted towards the major ones.	`currency`	—
`invoice`	An invoice - number, dates, net, tax and gross, all decimal.	`invoice_number, issued_on, net_amount, tax_amount, gross_amount, paid`	`currency`
`price`	A price with a long tail, as a decimal.	`price`	—
`product`	A catalogue item - identifier, name, category, price, stock.	`name, category, stock_on_hand, discontinued`	`sku, price, currency`
`sku`	A stock-keeping unit, unique per record.	`sku`	—
`transaction`	A payment - amount, method, status, authorisation code.	`transaction_id, occurred_at, amount, method, status, authorisation_code`	`currency`
OPERATIONAL			
`comment`	A short human note about a record. *Needs a language-model provider, or the placeholder is used.*	`comment`	—
`priority`	A priority, and the severity it implies.	`priority`	—

RECIPE	WHAT IT IS	FIELDS	INCLUDES
`status`	A workflow status, weighted like a real queue.	`status`	—
`ticket`	A support ticket - identifier, subject, status, priority, times. *Needs a language-model provider for the summary, or the placeholder is used.*	`ticket_id, channel, category, summary`	`status, priority, timestamp`
`timestamp`	Created and updated times, in order.	`created_at, updated_at`	—

APPENDIX

C

Transform operations

Sixteen operations, available in `patches:`, in the `transform` generator, and on the command line as `--set FIELD=OP`. Several are joined with `|`.

OPERATIONS

OPERATION	ALIASES	EFFECT
`lowercase`	`lower`	Case
`uppercase`	`upper`	Case
`title`	—	Title case
`strip`	`trim`	Leading and trailing whitespace
`slug`	—	Lowercase, hyphenated
`normalize`	—	Collapse whitespace, normalise Unicode to NFKC
`truncate:N`	`trunc`	At most N characters. Default 50
`mask:N`	`redact`	All but the last N characters become `*`. Default 4
`hash:N`	—	A stable BLAKE2b digest, N hex characters. Default 32
`round:N`	—	N decimal places; a `Decimal` stays a `Decimal`
`add_noise:N`	`noise`	Jitter a number by up to N per cent, deterministically
`format_date:FMT`	`date_format`	`strftime` a date or datetime. Default `%Y-%m-%d`
`encode:KIND`	`b64`	`base64`, `hex`, `url` or `json`
`compress:N`	—	Deflate and return base64-encoded
`decompress`	—	The inverse, so a transformed file can be read back
`nullify`	`null`	Drop the value entirely

Every operation is deterministic, including `add_noise`. Its jitter is derived by hashing the value rather than drawn from a generator, because an operation that reached for a random number would make a transformed dataset unreproducible and would change the file every time it ran.

An inapplicable operation raises. Rounding a name or formatting something that is not a date stops the pass rather than passing the value through: a

masking run that quietly skipped half a column is worse than one that failed. A null short-circuits, because deliberate nulls are what a chaos preset put there.

APPENDIX

D

Expressions, templates and filters

One evaluator, used in four places: the `expression` generator, simulation `state.update`, scenario effects beginning `=`, and the `--where` family of filters.

D.1 Functions

EXPRESSION FUNCTIONS

FUNCTION	DOES
`lower(s) · upper(s) · title(s)`	Case
`strip(s)`	Trim whitespace
`slug(s)`	Lowercase, hyphenated
`len(s)`	Length
`str(x) · int(x) · float(x) · bool(x)`	Conversion
`round(x, n) · abs(x)`	Arithmetic
`min(…) · max(…) · sum(…)`	Aggregation over arguments
`concat(a, b, …) · join(sep, …)`	Text assembly
`replace(s, old, new)`	Substitution
`substr(s, start, end)`	Slice
`coalesce(a, b, …)`	First non-null
`iif(cond, a, b)`	Conditional. Named `iif` because `if` is a Python keyword
`when(cond, a)`	Conditional with no else
`hash(s)`	A stable digest
`year(d) · month(d) · day(d)`	Date parts
`format_date(d, fmt)`	`strftime`
`contains(s, sub) · startswith(s, p)`	Tests

D.2 Template filters

FILTER	EFFECT
`lower · upper · title`	Case
`strip · nospace`	Trim, or remove all spaces

FILTER	EFFECT
`slug`	Lowercase, hyphenated, ASCII
`initial`	First character
`trunc:N`	At most N characters
`pad:N`	Zero-pad to width N

D.3 What is not available, on purpose

No `eval`, no imports, no attribute access beyond dotted record lookups, no comprehensions, no lambdas, no assignment. The evaluator walks a parsed syntax tree and refuses any node type not on the allow-list. A project file is something people send each other, and this is the boundary that makes opening one safe.

APPENDIX

E

Schema keys

Every key the schema language accepts, in one place, with the chapter that explains it.

TOP LEVEL

KEY	CONTAINS
`project`	`name` · `description` · `version` · `seed` · `locale` · `profile` · `provenance`
`entities`	A mapping of entity name to `count` · `description` · `primary_key` · `seed` · `tags` · `recipes` · `simulation` · `fields`
`relationships`	A list of `from` · `to` · `cardinality` · `field` · `required`
`providers`	A mapping of id to `type` · `adapter` · `base_url` · `model` · `secret` · `concurrency` · `timeout_seconds` · `options`
`timeline`	`start` · `end` · `shape` · `holidays` · `holiday_weight` · `months` · `spikes` · `growth`
`scenarios`	A mapping of name to `description` · `applies_to` · `affects_fraction` · `parameters`
`chaos`	`preset` plus any of the seven damage rates
`quality`	`duplication:` with `max_exact` · `max_near` · `fields` · `methods` · `similarity` · `shingle` · `window` · `error_rate` · `enabled`
`recipes`	The project's own fragments: `description` · `includes` · `fields`
`requires`	`plugins`
`patches`	A mapping of name to `description` · `entity` · `where` · `set` · `drop` · `keep`
`outputs`	A mapping of name to `format` · `path` · `entities` · `partition_by` · `options`, selected with `--output-profile`

FIELD

KEY	MEANING
`type`	One of 28 types
`semantic`	What the field means, in natural language
`description`	Documentation; falls back to `semantic`
`generator`	A name, or a mapping with `type` and options
`primary_key` · `unique`	Identity and uniqueness
`nullable` · `null_probability`	Nullability; `true` alone means 5%
`depends_on` · `context`	Ordering, and what a generator may consult
`constraints`	`min` · `max` · `min_length` · `max_length` · `pattern` · `enum` · `forbidden` · `multiple_of` · `precision`

KEY	MEANING
`examples · tone · locale`	Hints for generators that can use them
`anything else`	A generator option

SIMULATION

KEY	MEANING
`subject`	Whose events these are
`distribution · skew`	How events are allocated across subjects: `uniform`, `skewed`, `zipf`
`minimum`	Events every subject gets before the rest are allocated
`state`	A mapping of variable to `initial · update · min · max · precision`

APPENDIX

F

Command synopsis

```
cacophony validate  PROJECT [--seed N]
cacophony lint      PROJECT [--strict]
cacophony plan      PROJECT [--seed N] [--json] [--batch-size N] [--workers N]
cacophony preview   PROJECT [-e ENTITY] [-n N] [-c a,b,c] [--offset N] [--json]
                            [--isolate] [--cache MODE] [--cache-path FILE] [--seed N]
cacophony generate  PROJECT [-n N | -n ENTITY=N] [--seed N] [-o FORMAT] [-d DIR]
                            [--output-profile NAME] [-e ENTITY]
                            [--batch-size N] [--workers N] [--provenance MODE]
                            [--on-failure POLICY] [--drop-invalid] [--no-validate]
                            [--edge-cases FRACTION] [--edge-categories a,b,c]
                            [--cache MODE] [--cache-path FILE] [-w WORLD]
                            [--assets-dir DIR] [--regenerate-assets]
                            [--llm-batch-size N] [--checkpoint-every N]
                            [--store FILE] [--no-history]
                            [--log-level LEVEL] [--log-format text|json]
cacophony resume    [RUN] [-p PROJECT] [--store FILE] [--log-level LEVEL]
cacophony runs      [-p PROJECT] [--store FILE] [--state STATE] [--limit N] [--json]
cacophony run       RUN [-p PROJECT] [--store FILE] [--events N] [--json]
cacophony regenerate PROJECT -r N|N-M [-e ENTITY] [-c a,b,c] [--json] [--seed N]

cacophony serve     [--host H] [--port P] [--store FILE] [-p PROJECT]
                            [--studio DIR] [--allow-root DIR]... [--insecure]
                            [--reload] [--log-level LEVEL]
cacophony desktop   [--store FILE] [-p PROJECT] [--studio DIR] [--host H]
                            [--port N] [--no-token] [--keep-running] [--log-level L]

cacophony generators [--json]
cacophony recipes   [-p PROJECT] [--show NAME] [--group NAME] [--json]
cacophony plugins   [--show NAME] [--json]
cacophony providers [PROJECT] [--test]
cacophony models    PROJECT [-p PROVIDER]
cacophony prompt    PROJECT [-e ENTITY] [--batch-size N] [--schema]
cacophony benchmark PROJECT -m MODEL,MODEL [-e ENTITY] [-n N] [-p PROVIDER]
                            [--sort-by FIELD] [--json] [--seed N]
cacophony begin     "DESCRIPTION" [-o SCHEMA] [-d DIR] [-f FORMAT] [--scale N]
                            [--seed N] [-e] [-y] [--force] [--providers PROJECT]
                            [--adapter NAME] [--url URL] [-m MODEL] [--store FILE]
cacophony propose   "DESCRIPTION" [-o FILE] [--providers PROJECT] [--adapter NAME]
                            [--url URL] [-m MODEL] [--scale N] [--seed N] [--force]
cacophony worlds    PROJECT [--create NAME] [--show NAME] [--delete NAME]
cacophony version

cacophony stream    PROJECT [-r ENTITY=RATE]... [-t DESTINATION]... [-s SECONDS]
                            [-n N] [--from N] [--batch-size N] [--flush SECONDS]
                            [--follow-shape] [--historical] [--validate]
                            [--scenario-cycle SECONDS] [--on-error POLICY]
                            [--seed N] [-q]
cacophony cluster   PROJECT [-o DIR] [-w WORKERS] [-f FORMAT] [-n N]
                            [--shard-size N] [--batch-size N] [--join|--no-join]
                            [--assets-dir DIR] [--seed N] [-q]
cacophony controller PROJECT [--host H] [--port P] [--shard-size N]
                            [--lease-seconds S] [--max-attempts N] [-n N]
                            [--seed N] [--log-level LEVEL]
```

```
cacophony worker    PROJECT -c URL [-o DIR] [-f FORMAT] [--name ID]
                            [--capabilities LIST] [--concurrency N] [--batch-size N]
                            [-n N] [--assets-dir DIR] [--idle-timeout S] [--seed N]

cacophony transform SOURCE [-s 'FIELD=OP']... [-w EXPR] [--drop-where EXPR]
                            [--keep-where EXPR] [-o FILE | --in-place] [-f FORMAT]
                            [-p PROJECT] [-e ENTITY] [--record-as NAME]
                            [--force] [--json]
cacophony bundle export  PROJECT [-o FILE] [--force]
cacophony bundle inspect FILE [--json]
cacophony bundle import  FILE -d DIR [--force]
```

APPENDIX

G

Exit codes and the environment

EXIT CODES

CODE	MEANING
0	Success
1	Lint errors
2	Bad schema or bad arguments
3	Generation failure
4	Run cancelled rather than failed

ENVIRONMENT VARIABLES

VARIABLE	EFFECT
CACOPHONY_SECRET_<ID>	Resolves a provider's logical secret
CACOPHONY_TOKEN	Required to serve on any interface but loopback; guards `/api`
CACOPHONY_CONTROLLER_TOKEN	Shared token for distributed generation
CACOPHONY_NO_PLUGINS	`1` skips plugin loading
CACOPHONY_BACKEND	Which backend the desktop shell spawns
CACOPHONY_TEST_OLLAMA	Base URLs for the live provider contract tests, which are skipped when unset
CACOPHONY_TEST_INVOKEAI	
CACOPHONY_TEST_PIPER	

APPENDIX

H

Glossary

asset

A generated file — image, audio or document — written under `assets/` with a line in the manifest tying it to its record.

attainment

In a stream, the achieved rate as a fraction of the requested one, integrated over time.

bundle

A `.cacophony` file: a project and everything it references, with a hash manifest.

capability

Something a shard needs or a worker has: `deterministic`, `language_model`, `image`, `speech`, `document`. Never declared; always derived.

chaos

Deliberate damage: data the schema forbids, injected at a stated rate.

checkpoint

How many records a job has finished. One integer, because seeds come from positions.

cost class

What a generator costs to run: `cpu`, `llm` or `gpu`. Used by the scheduler and by the distributed controller.

edge case

A legal value that naive code mishandles anyway. Distinct from chaos.

entity

A record type: a table, a document collection, a log stream.

lease

A time-limited grant of one shard to one worker, renewed while it works.

patch rule

An edit recorded in the project rather than applied to a file, so it survives regeneration.

plan

The compiled result: entities in dependency order, and per field its generator, cost, determinism and dependencies.

provenance

Recorded facts about how data came to exist, at five levels of detail.

recipe

A named group of fields, expanded into an entity before anything else looks at the schema.

run

One execution of `generate`, recorded in the store with its jobs, checkpoints and events.

scenario

A behavioural overlay applied to a fraction of subjects over a window of their history.

seed tree

The hierarchy from which every value's randomness is derived by position rather than by sequence.

shard

A contiguous index range, generated independently and joined without a seam.

simulation

An entity declared to be events belonging to subjects, with state folded over each subject's block.

subject

Whoever an event belongs to: a user, an account, a device.

world

A named seed plus the schema it goes with, checked when used.

APPENDIX

I

Lint rules

NINETEEN RULES AT THREE SEVERITIES. `--strict` MAKES WARNINGS FATAL.

RULES

CODE	SEVERITY	FIRES WHEN
`unique-exhaustion`	error	A field is marked unique but its generator cannot produce that many distinct values
`reference-to-empty-entity`	error	A reference points at an entity with no records
`unique-reference-overflow`	error	A unique reference needs more parents than exist
`missing-provider`	error	A field names a provider that is not configured
`empty-entity`	warning	`count` is 0, so no records will be produced
`self-reference`	warning	An entity references itself; check it points backwards and is nullable
`llm-without-semantics`	warning	A model field has no `semantic`, so the prompt has nothing to say
`bulk-media`	warning	Media generation at a record count that will take a very long time
`bulk-llm`	warning	The same, for model calls
`unrealistic-distribution`	warning	Distribution parameters that will produce implausible values
`reference-to-nonunique-key`	warning	A reference points at a column that is not unique, so the join is ambiguous
`chaos-referential`	warning	Referential chaos with an output that enforces foreign keys
`no-primary-key`	info	No primary key is declared and the entity has more than one record
`unbounded-text`	info	A long text field has no `max_length`, so prose size is unpredictable
`mostly-null`	info	Half or more of the values will be null
`uniform-reference`	info	A reference is uniform, which almost nothing in the world is
`unused-provider`	info	A provider is configured but never used
`chaos-absolute`	info	The `absolute` preset damages so much that most records are unusable

APPENDIX

J

Where the design document went

Cacophony was built from a 112-section design document. This maps its numbered sections to the chapters here, for anyone reading both.

> **AND WHAT WAS NOT BUILT**
>
> This appendix says where each subject is *explained*. It does not say whether it is *finished*. That is `docs/CONFORMANCE.md` : every section with one of four states — built, partial, deferred, declined — generated from a data file and checked on every push, so it cannot drift the way a hand-written claim does. 6 things the document asks for were considered and refused, each with its reasoning recorded: script (§8), embedding-based detection (§59), detecting suspiciously realistic public identities (§61), claiming the output is privacy-preserving (§61), semantic quality scoring (§67) and GPU generation time (§69). Both of those sentences are generated from the matrix now, because the last two refusals were added to it and not to this paragraph.

SECTIONS	SUBJECT	HERE
8, 12, 13, 66	Generators, prompt compilation, model trust	Chapters 11–16
15, 25, 26	References, timelines, partitioned folds	Chapters 17–18
16, 17	Worlds and scenarios	Chapters 19, 22
18, 20, 23, 19, 81	Images, speech, documents, asset provenance	Chapter 16
24, 78, 79	Chaos and edge cases	Chapters 20, 34
28, 31, 32, 33, 37	Planning, streaming writes, checkpoints, formats	Chapters 5, 10, 27
35, 94	Live streams	Chapter 29
36, 43	Providers and adapters	Chapter 26
41	The desktop application	Chapter 39
44	Plugins	Chapter 40
45–56, 48, 73, 74	The API, the Studio, schema revisions	Chapters 37–38
57, 58, 59	Validation and duplication	Chapters 32–33
62, 63, 87	Safe identifiers, credentials, provenance retention	Chapter 6
67	Model benchmarking	Chapter 35
72	Bundles	Chapter 23
75	Hierarchical positional seeds	Chapter 4
80, 106	Recipes and the catalogue	Chapter 21, appendix B

SECTIONS	SUBJECT	HERE
84, 95	Distributed generation and leases	Chapter 30
102	Linting	Appendix I
104, 105	Patch rules and transformations	Chapter 36
110	One sentence to a world	Chapter 28

APPENDIX

K

Index

Generators, commands, schema keys, operations and terms, with the page each is defined on.